



南開大學
Nankai University

计算机学院
软件工程实验报告

第三次作业-软件设计

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2026 年 7 月 14 日

目 录

1	引言	3
1.1	编写目的	3
1.2	项目背景	3
1.3	设计方法与工具	3
2	用例图	4
2.1	用例图简介	4
2.2	用例图设计思路	5
2.3	系统参与者分析	5
2.4	用例图构建	6
2.5	主要用例说明	10
3	活动图	10
3.1	活动图简介	10
3.2	活动图设计思路	11
3.3	核心业务流程分析	11
3.4	活动图构建	12
3.5	活动流程说明	16
4	类图	16
4.1	类图简介	16
4.2	类图设计思路	17
4.3	核心类与职责划分	17
4.4	类图构建	18
4.5	类间关系说明	20
5	顺序图	20
5.1	顺序图简介	20
5.2	顺序图设计思路	21
5.3	典型交互场景选择	21
5.4	顺序图构建	22
5.5	消息传递过程说明	25
6	协作图	26
6.1	协作图简介	26
6.2	协作图设计思路	26
6.3	协作对象分析	27
6.4	协作图构建	27
6.5	对象协作关系说明	29

7 状态图	30
7.1 状态图简介	30
7.2 状态图设计思路	30
7.3 生成任务生命周期分析	30
7.4 状态图构建	31
7.5 状态迁移说明	33
8 构件图	33
8.1 构件图简介	33
8.2 构件图设计思路	34
8.3 系统构件划分	34
8.4 构件图构建	35
8.5 构件依赖关系说明	36
9 部署图	37
9.1 部署图简介	37
9.2 部署图设计思路	37
9.3 运行节点划分	37
9.4 部署图构建	38
9.5 部署结构说明	39
10 总结	40

1 引言

1.1 编写目的

在前期进行了需求分析的基础上，我们还需要进一步明确智能城市生成系统的静态结构、动态行为、对象关系、构件划分和部署方式。结合期末团队作业的总体方向，我认为本系统不应该设计成一个脱离插件环境独立运行的传统建模平台，而是由“主系统原型 + Blender 城市场景生成插件系统”共同构成。其中，主系统原型负责多角色登录、权限控制、项目管理、模板管理和任务调度，而 Blender 插件系统负责道路、建筑、绿化、水体以及动态对象的生成与精细编辑，两者协同完成城市空间场景的快速构建与后续调整。

设计上，我认为这套系统至少同时面临三个问题。首先，城市场景中的对象类型较多，而且道路网、建筑群、植被层、水体和交通设施之间还存在明显的空间依赖，仅靠文字很难准确说明它们的组织关系。其次，主系统、智能交互服务与 Blender 插件系统之间的调用链较长，许多操作都需要多个模块协同完成。最后，生成任务本身还涉及失败恢复、任务重试、版本沉淀和结果回滚等工程问题。基于这些特点，我们分别使用用例图、活动图、类图、顺序图、协作图、状态图、构件图和部署图，从不同视角说明我们的系统设计。

1.2 项目背景

随着人工智能技术、大语言模型和三维建模技术的发展，城市建模已经不再只是单纯的几何搭建问题，而是逐渐转向“快速生成、持续调整、反复比较”的综合过程。无论是城市规划展示、数字孪生，还是游戏与影视虚拟场景、教学演示，都希望在较短时间内得到结构合理、视觉完整、便于修改并能够支持后续分析的城市模型，而传统依赖人工逐项搭建的方式在效率、复用性和迭代速度上都存在明显局限，难以适应这类需求。

基于这一背景，智能城市生成系统需要把资源管理、模板驱动生成、空间布局编辑、自然语言交互、插件自动调用、动态仿真和结果导出组织到同一条工作流中。系统整体分为主系统原型和 Blender 插件系统两个部分：前者负责多角色登录、权限区分、项目组织、模板管理和任务调度，后者负责道路、建筑、绿化、水体以及动态对象的具体生成与精细编辑。这样的划分既保证了主系统能够统一管理任务和流程，也保证了插件系统能够专注于三维场景内容本身的生成效果。

在实际使用过程中，不同角色对系统的关注点并不相同。场景建模师需要借助模板、资源和插件能力尽快完成初版场景，并在此基础上继续调整细节；行业分析师更关注生成结果是否符合业务目标，是否能够支撑后续仿真、比较和展示；系统管理员则需要维护模板、权限、日志和版本，保证整套生成流程稳定可控。因此，系统设计不能只关注生成一个城市模型，还必须同时考虑协作、校验、追踪和维护等配套能力。

1.3 设计方法与工具

系统设计采用 UML 统一建模语言，从功能边界、流程组织、对象关系、交互方式和运行环境几个方面展开分析。这里选取用例图、活动图、类图、顺序图、协作图、状态图、构件图和部署图八类图示，对系统进行分层描述，其中不同图示关注的重点并不相同，只有结合起来阅读，才能较完整地理解整套设计。

为保证各部分设计前后一致，我们在建模过程中主要遵循以下原则：

- 面向角色划分功能边界，避免将所有功能堆叠到单一用户视角中；

- 面向任务组织业务流程，突出从项目创建到场景生成、仿真预览和结果导出的闭环；
- 面向对象抽象核心数据结构，使项目、场景、资源、模板、任务、插件和版本具有清晰关系；
- 面向构件划分系统责任，保证实现阶段可以独立扩展资源服务、插件服务、智能交互服务和仿真服务；
- 面向部署考虑运行环境，兼顾本地演示、轻量化部署和分布式扩展。

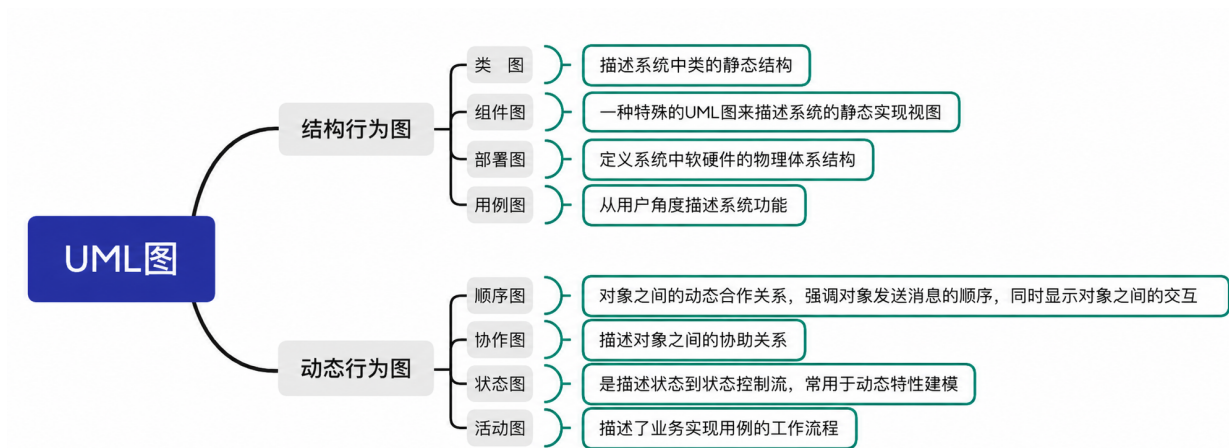


图 1: 常见的 8 类 UML 图

2 用例图

2.1 用例图简介

用例图（Use Case Diagram）用于说明系统对外提供哪些功能，以及这些功能分别由谁发起。它关注的不是内部实现细节，而是系统边界、参与者职责和主要业务能力之间的对应关系。对智能城市生成系统来说，用例图首先要回答的不是页面上有哪些菜单，而是场景建模师、行业分析师和系统管理员分别需要完成哪些工作，哪些能力属于主系统，哪些能力依赖外部服务。

就组成元素而言，用例图主要包括参与者、用例、系统边界以及用例之间的关系。参与者可以是人，也可以是外部服务；用例表示系统能够提供的一项完整功能；系统边界用于区分系统职责与外部环境；包含、扩展和泛化等关系则用来说明不同功能之间是必然关联还是条件触发。在本系统中，项目管理、资源导入、模板生成、版本比较和任务重试属于系统内部职责，而智能交互服务、Blender 插件系统和对象存储则属于外部能力接入。

在设计本节用例图时，还需要特别注意几类关键关系。关联关系说明某个角色能够直接发起哪些操作；包含关系说明一个主用例在执行时必然会带出哪些子过程，例如提交生成任务必然伴随参数校验、资源校验和插件调用；扩展关系则用于表示只在特定条件下发生的附加动作，例如高影响的自然语言修改需要先进行用户确认。由于智能城市生成系统的业务链较长，如果只是单纯地把生成、编辑、导出这类结果动作画出来，很多真正影响系统可用性的环节就会被忽略，因此校验、确认、回写、比较和回滚都应在用例层面体现出来。

表 1: 用例图常用元素说明

元素	图中含义	在本系统中的示例
参与者	与系统发生交互的人或外部系统	场景建模师、行业分析师、系统管理员、Blender 插件服务
用例	系统对外提供的一项完整服务	创建项目、提交生成任务、导出结果、查看日志
系统边界	标识哪些功能属于当前系统范围	智能城市生成系统矩形边界内的功能属于本系统设计范围
包含关系	主用例执行时必然发生的子功能	生成场景包含资源校验、参数校验和插件调用
扩展关系	满足特定条件时才发生的附加功能	高影响自然语言修改需要扩展出用户确认

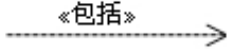
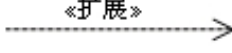
关系类型	说明	表示符号
关联	参与者与用例间的关系	
泛化	参与者之间或用例之间的关系	
包含	用例之间的关系	
扩展	用例之间的关系	

图 2: 用例图常见关系类型及表示符号

2.2 用例图设计思路

智能城市生成系统既有面向场景生产的功能，也有面向分析、治理和运维的功能。如果把所有内容都压缩到一张图里，参与者和用例之间的连线会非常密集，主次也会变得不清楚。因此我们在这里先给出总体用例图，再按业务边界继续拆分为项目与资源管理、场景生成与编辑、智能交互与插件协同，以及后台治理与日志监控几个部分。

采用这种拆分方式有两个直接好处。首先，总体图可以先把系统面对哪些角色、依赖哪些外部能力讲清楚；其次，拆开的子图能够把每条业务线中的关键动作单独展开，不至于让建模流程、分析流程和管理员流程混在一起。对智能城市生成系统来说，这样的处理也更贴近实际使用方式，因为城市内容生产链和后台治理链本来就是两类关注点不同的工作过程。

2.3 系统参与者分析

系统中的主要参与者可以分为三类。其中，场景建模师直接承担城市内容生产，是最核心的使用者；行业分析师主要围绕结果核对、方案比较和仿真验证展开工作；系统管理员则负责模板、权限、插件和日志等后台治理事项。三类角色虽然共用同一套系统，但进入系统后的关注重点并不相同，这也是我们后续进行权限划分和功能拆分的重要依据。

具体来说，场景建模师通常从项目创建、资源导入和模板选择开始，随后配置参数、提交生成任务，并在生成结果基础上继续做局部调整和成果导出；行业分析师更多关注不同版本之间的差异，以及结果是否满足业务目标和仿真要求；系统管理员则需要保证整条生成链路稳定运行，包括模板审核、插件配置、任务排障、日志监控和版本恢复等工作。除这些直接用户外，系统还依赖 Blender 插件系

我们把图 3 作为整套系统的总览用例图，其中最大的矩形边界表示智能城市生成系统，边界内部再按照业务性质划分为项目与资源、原型系统入口、场景生产、智能协同和分析与治理五个功能区域。左侧三个直接参与者分别是场景建模师、行业分析师和系统管理员，右侧两个外部参与者分别是大语言模型服务和 Blender 插件服务。这样的布置说明本系统既有面向普通用户的业务入口，也有需要外部智能服务和三维插件环境支撑的生成能力。场景建模师主要关联维护资源引用、导入底图与资源、创建与管理项目、进入 Blender 插件系统、多角色登录、生成与编辑场景、导出结果与保存版本和动态仿真预览等用例，说明其工作重心是从项目准备到场景生产再到成果输出的完整链路。行业分析师主要关联动态仿真预览、比较方案版本和核对生成结果，说明其不直接承担场景搭建，而是围绕生成成果进行审查、比较和评价。系统管理员则连接模板与插件治理、权限与日志管理等用例，负责后台配置、审计和运行维护。

大语言模型服务则与自然语言控制、草图识别关联，Blender 插件服务与插件调用关联，表示这两类能力位于系统边界之外，需要通过接口接入。图中虚线的 <<include>> 关系表明若干用例是主流程中的必经子过程，例如生成与编辑场景需要包含配置参数和选择模板，插件调用又是智能协同和场景生成能够落地到 Blender 环境的必要步骤。因此，我们在总体用例设计中并不是简单罗列功能，而是在整体层面说明不同角色、系统内部能力和外部服务之间的边界关系。

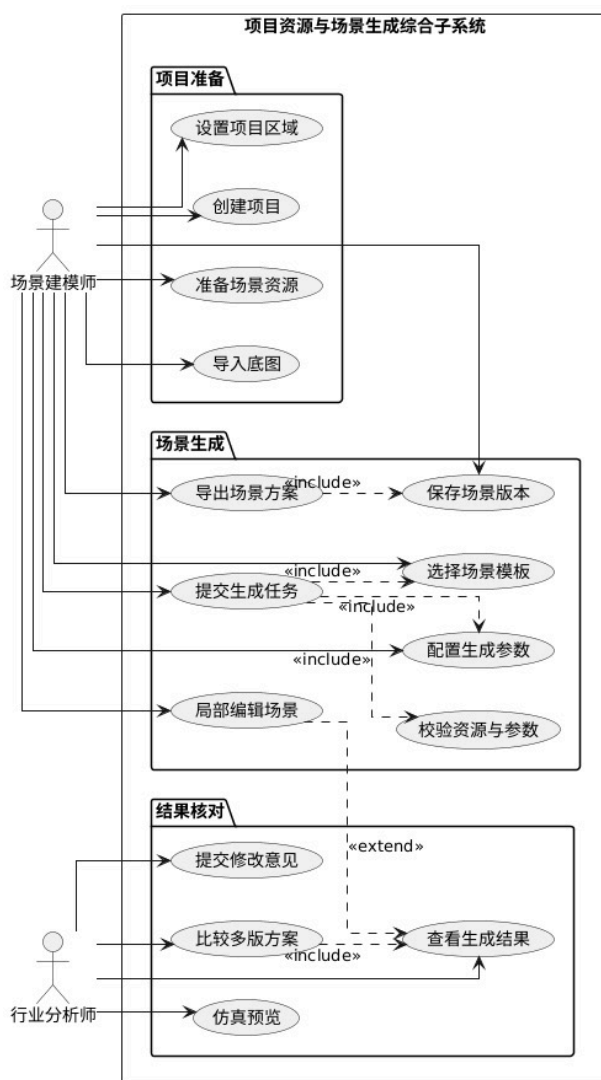


图 4: 项目资源与场景生成综合用例图

我们在图 4 中对总体用例图里的项目与资源和场景生产部分作进一步展开，并将系统边界命名为项目资源与场景生成综合子系统。左侧参与者包括场景建模师和行业分析师。上方项目准备包中包含设置项目区域、创建项目、准备场景资源和导入底图四个用例，其中设置项目区域用于确定城市生成的空间范围，准备场景资源和导入底图用于提供道路、建筑、绿化、水体等生成所需的基础输入。场景建模师与这些用例相连，说明项目创建和资源准备是建模师发起生成任务前必须完成的操作。

中部场景生成包描述城市内容生产的核心用例。其中，提交生成任务并不是孤立动作，它通过 `<<include>>` 关系包含选择场景模板、配置生成参数和校验资源与参数，表示只有模板、参数和资源都满足要求后，系统才会进入实际生成流程。导出场景方案通过 `<<include>>` 指向保存场景版本，说明系统在输出成果前必须先固化当前版本，避免导出文件与历史场景状态脱节。局部编辑场景被单独列出，表示自动生成之后仍允许建模师对局部道路、建筑高度、绿化密度或水体边界进行人工调整。

下方结果核对包则体现了行业分析师的参与方式。行业分析师可以查看生成结果、比较多版方案、仿真预览和提交修改意见。其中比较多版方案包含查看生成结果，说明版本比较必须基于具体生成结果展开；提交修改意见对查看生成结果形成 `<<extend>>` 关系，表示只有在分析师查看结果后发现问题或提出优化建议时，才会追加修改意见。通过这些关系，我们就成功地把从项目准备、任务生成、版本保存到结果核对的连续闭环表达了出来。

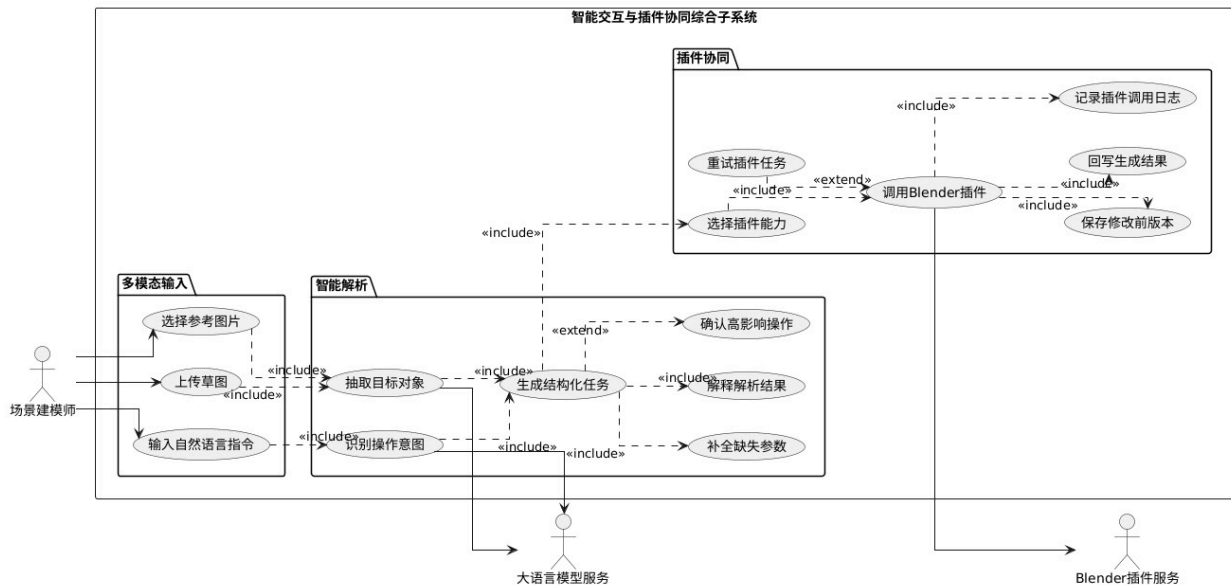


图 5: 智能交互与插件协同综合用例图

我们在图 5 中展开智能交互与插件协同综合子系统。左侧的多模态输入包中包含选择参考图片、上传草图和输入自然语言指令三个用例，表示场景建模师不只通过表单参数控制生成，也可以通过图片、草图和自然语言表达意图。中部智能解析包将用户输入继续拆解为抽取目标对象、识别操作意图、补全缺失参数、解释解析结果、生成结构化任务和确认高影响操作等用例。图中多条 `<<include>>` 关系说明自然语言或草图输入不能直接变成插件命令，而必须先识别对象、动作和参数，再组装成结构化任务。

这里最关键的部分是确认高影响操作与生成结构化任务这两个用例之间的 `<<extend>>` 关系。它表示并非每一次智能解析都需要用户确认，只有当系统判断操作会造成大范围修改时，例如重排道路网、批量调整建筑高度或整体替换绿化层，才会扩展出确认步骤。右上方插件协同包则进一步描述了任务落地过程：选择插件能力通过 `<<include>>` 进入调用 Blender 插件，调用 Blender 插件又包含记录插件调用日志、回写生成结果和保存修改前版本等动作。这里的保存修改前版本说明了我们的系统

在执行智能修改前会保留原始场景状态，便于失败恢复和人工回滚。

我们还在图中把大语言模型服务和 Blender 插件服务画在系统边界外部，其中大语言模型服务参与抽取目标对象和识别操作意图，但不直接修改场景；Blender 插件服务执行具体三维生成命令，但生成结果需要回写到主系统后才成为正式版本。因此，我们在这部分用例设计中强调“输入解析—任务结构化—风险确认—插件执行—结果回写”的安全链路，而不是让外部智能服务直接接管场景数据。

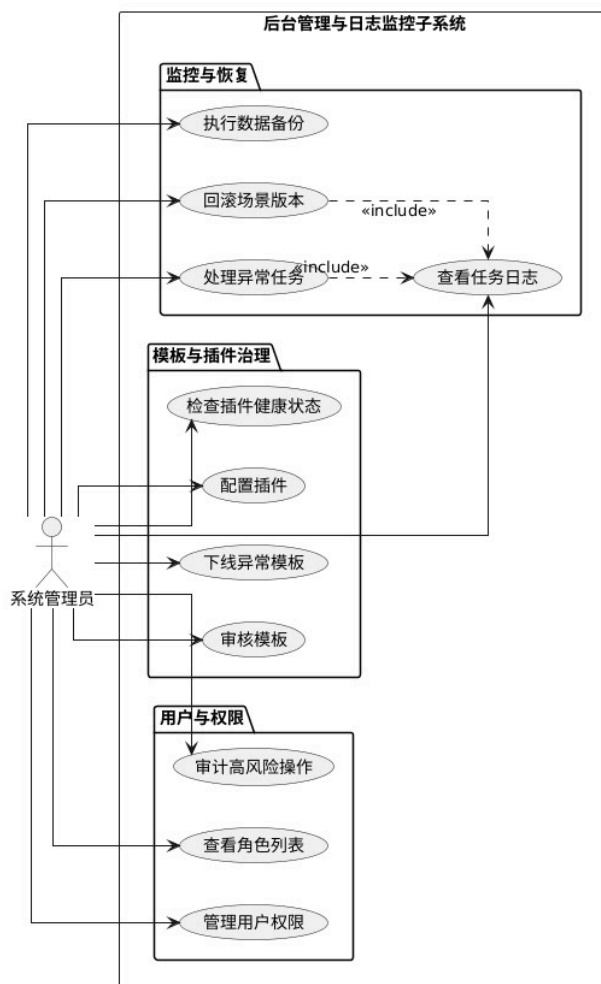


图 6: 后台管理与日志监控子系统用例图

我们用图 6 描述系统管理员使用的后台管理与日志监控子系统，并把功能划分为监控与恢复、模板与插件治理、用户与权限三个区域，说明后台管理不是单一的用户管理页面，而是同时覆盖运行监控、模板治理、插件配置和权限审计，并且系统管理员与全部后台用例相连，表示这些操作均需要管理员权限。

在监控与恢复区域中，执行数据备份用于周期性保存关键数据，回滚场景版本用于在错误生成或误操作之后恢复历史结果，处理异常任务用于处理失败、阻塞或超时的生成任务。图中回滚场景版本和处理异常任务都通过 <<include>> 指向查看任务日志，说明管理员在做恢复或排障之前必须先读取日志，确认异常发生的任务、插件、模板和资源环境。该设计避免管理员在没有上下文的情况下直接执行回滚或重试。

在模板与插件治理区域中，检查插件健康状态表示系统需要检测插件是否可用，配置插件用于调整插件端点或能力参数，下线异常模板用于临时阻止有问题的模板继续被项目使用，审核模板用于控

制公共模板的发布质量。用户与权限区域中的审计高风险操作、查看角色列表和管理用户权限对应多人协作场景下的访问控制。由于模板和插件通常会被多个项目复用，所以我们在管理员相关用例中强调异常追踪、影响范围限制和版本恢复，而不仅仅停留在账号管理。

2.5 主要用例说明

智能城市生成系统的用例具有明显的任务链特征。场景建模师通常先创建项目，再导入底图、资源和模板，随后配置生成参数并提交任务。系统完成初始场景生成后，场景建模师还需要继续做局部编辑和结果导出。行业分析师的用例集中在结果核对，包括查看场景版本、检查参数配置、比较不同生成方案和查看动态仿真结果。系统管理员的用例则围绕治理和维护展开，包括用户管理、模板审核、插件配置、异常处理、日志检索和版本恢复。

就用例之间的关系而言，模板驱动生成通常包含资源校验、参数配置和插件调用；自然语言修改场景则依赖智能解析和任务创建；结果导出一般包含版本保存。通过这些包含和依赖关系，可以避免把复杂功能写成孤立用例，从而更准确地反映系统真实业务过程。

进一步来看，用例图还反映出系统的权限边界。场景建模师可以直接操作场景和资源，但不应直接修改公共模板的发布状态；行业分析师可以查看和评价结果，但不直接执行高风险的场景覆盖操作；系统管理员可以处理模板、插件和日志问题，但通常不参与具体项目的建模细节。这样的边界划分有利于实现权限控制，也能减少多人协作时的误操作风险。

在工程实现阶段，用例图中的外部服务不应被设计成系统内部不可替换的一部分。大语言模型服务、草图识别服务和 Blender 插件服务都可能随着技术路线变化而替换，因此主系统更适合通过统一接口与它们通信。这样既保留智能化能力，也避免系统被某一个具体模型或插件绑定。

对于智能城市生成系统这一主题，用例图还有一个重要作用，就是把城市内容生产过程拆分为若干可管理的工作单元。例如，项目创建对应城市区域和业务目标的确定，资源导入对应地形、建筑、植被和交通设施等基础素材的准备，模板生成对应道路网、建筑群和景观层的批量搭建，仿真预览对应人流、车流和环境效果的验证，版本比较则对应方案迭代与成果评估。这样处理后，图中的每个用例都能与具体的城市生成活动相对应，而不是停留在抽象的软件功能名称上。

3 活动图

3.1 活动图简介

活动图 (Activity Diagram) 主要描述业务流程中各个活动节点的执行顺序、条件判断、分支合并和流程终止条件。与用例图相比，活动图更加关注一个用例如何完成，因此常用于分析复杂业务流程、异常处理过程和多角色协同过程。活动图中的活动节点表示具体操作，判断节点表示条件分支，开始和结束节点则表示流程边界。

在智能城市生成系统中，很多功能并不是单次按钮点击即可完成，而是由多个连续步骤构成。例如场景建模师生成街区场景时，需要先准备资源，再选择模板，接着配置参数、提交任务、等待插件执行，最后进行预览、精修和导出。活动图能够把这些步骤串联起来，同时展示资源缺失、参数异常和插件失败时系统如何处理。

活动图的优点在于能够同时表达正常流程和异常流程。对智能城市生成系统来说，异常流程并不是次要内容，因为插件失败、资源缺失、自然语言歧义和导出失败都可能在实际使用中出现，如果设计文档只描述成功路径，实现阶段就容易忽略错误提示、任务回退、版本保护等工程细节。因此我们

在活动图中保留多个判断节点，使流程不仅能说明系统如何完成任务，也能说明系统在遇到问题时如何保持可控。

表 3: 活动图常用元素说明

元素	图中含义	在本系统中的示例
开始节点	表示流程的起点	用户进入系统、创建项目、提交指令
活动节点	表示一个可执行步骤	导入资源、选择模板、调用插件、保存版本
判断节点	根据条件选择不同路径	资源是否完整、参数是否合法、插件是否成功
合并节点	多个分支重新汇合	异常处理后重新回到参数调整或任务提交
结束节点	表示流程结束	成果导出完成、任务取消、异常处理结束

3.2 活动图设计思路

活动图用于描述业务流程中的活动顺序、判断分支和并行关系。智能城市生成系统包含多个任务流程，其中最具有代表性的包括场景建模师快速搭建城市街区、行业分析师核对生成结果、系统管理员处理模板异常以及自然语言修改场景。我们本次的活动图设计就围绕这四条流程展开，可以较完整地覆盖生成前准备、生成中调度、生成后核对、异常时治理这四类典型情况。

3.3 核心业务流程分析

系统的核心业务流程可以概括为准备资源、选择模板、生成场景、检查结果、精修调整、保存导出。在这个过程中，系统需要对资源完整性、参数合法性、插件执行结果和版本保存状态进行持续检查。如果资源缺失、参数异常或插件执行失败，系统应给出明确提示并允许用户重试、回退或转入人工编辑流程。

表 4: 活动图覆盖的典型业务流程

流程名称	主要参与者	设计关注点
快速搭建城市街区	场景建模师	资源准备、模板生成、参数校验、人工精修与导出
核对生成结果	行业分析师	版本查看、参数核对、仿真结果分析、修改意见提交
处理模板异常	系统管理员	日志定位、模板下线、依赖检查、修复后重新发布
自然语言修改场景	场景建模师、智能交互服务	指令解析、影响范围确认、任务创建、结果回写

3.4 活动图构建

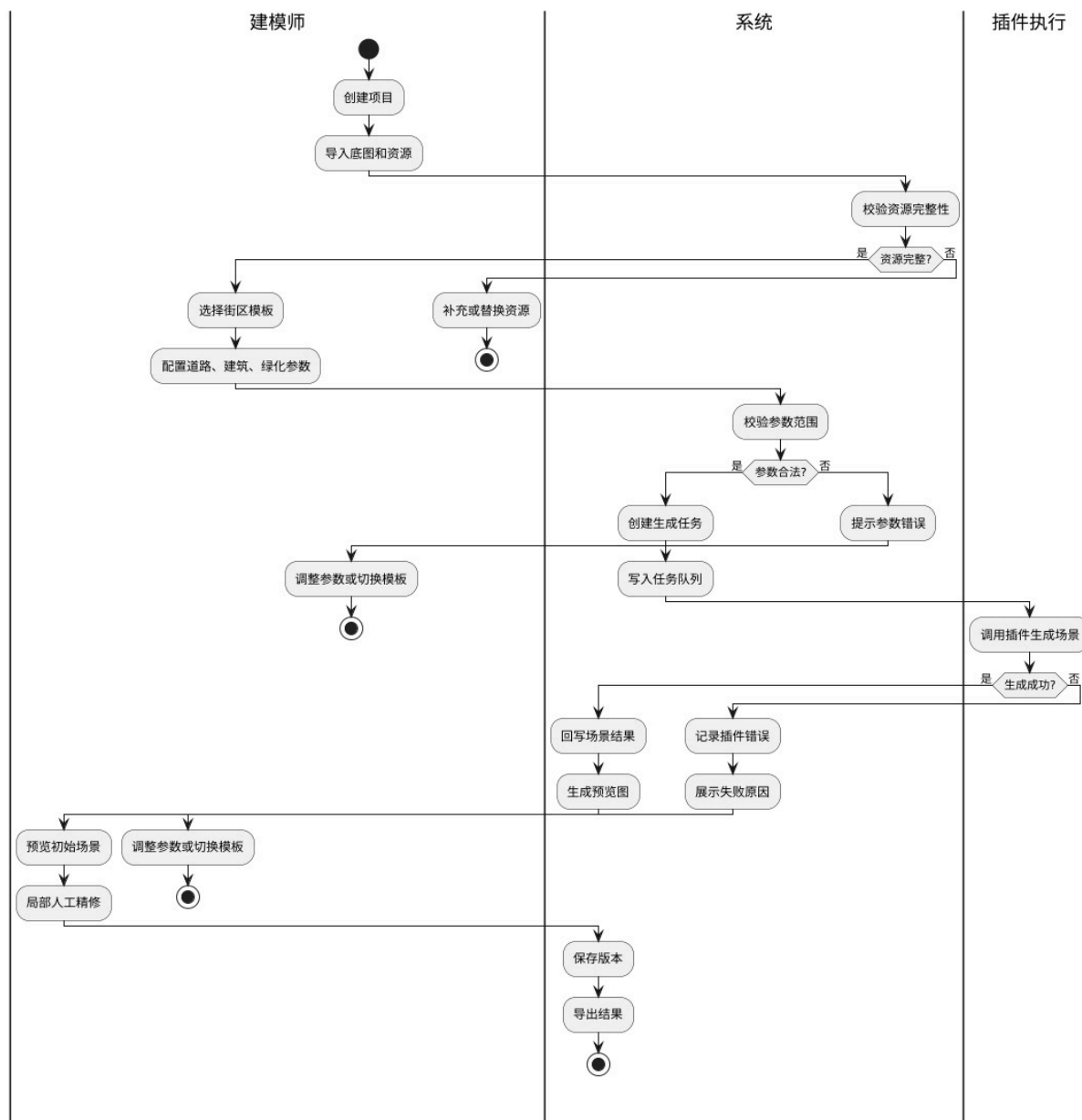


图 7: 场景建模师快速搭建城市街区活动图

我们在图 7 中采用建模师、系统、插件执行这三条泳道描述城市街区快速搭建流程。首先，建模师泳道从创建项目开始，随后执行导入底图和资源，表示用户先确定项目并提供地图、模型、材质等基础输入，接着该输入被传递到系统泳道中的校验资源完整性节点，系统继续通过“资源完整？”这个分支来进一步判断资源是否满足生成要求。如果判断结果为否的话，流程就回到建模师泳道中的补充或者替换资源，并在补充后结束当前异常分支；如果判断为是，流程才进入选择街区模板和配置道路、建筑、绿化参数。

参数配置完成后，系统执行校验参数范围，并通过“参数合法？”判断是否可以创建生成任务。若参数不合法，系统进入提示参数错误，流程返回建模师侧的调整参数或切换模板；若参数合法，系统创建生成任务并写入任务队列，随后进入插件执行泳道的调用插件生成场景。插件执行后通过“生成

成功?”进行分支判断：成功时，系统回写场景结果并生成预览图；失败时，系统记录插件错误并展示失败原因，再引导建模师调整参数或更换模板。

需要说明的是，我们在图中的后半部分并没有把“生成成功”作为终点，而是继续让建模师预览初始场景并进行局部人工精修，最后由系统执行保存版本和导出结果，这说明本系统中的插件生成只负责形成初始城市框架，而最终成果仍需要经过用户预览、人工修正和版本沉淀。这样的流程更符合城市建模任务的实际特点：自动生成提高效率，资源校验、参数校验和失败回退保证过程可控，人工精修则保证最终场景质量。

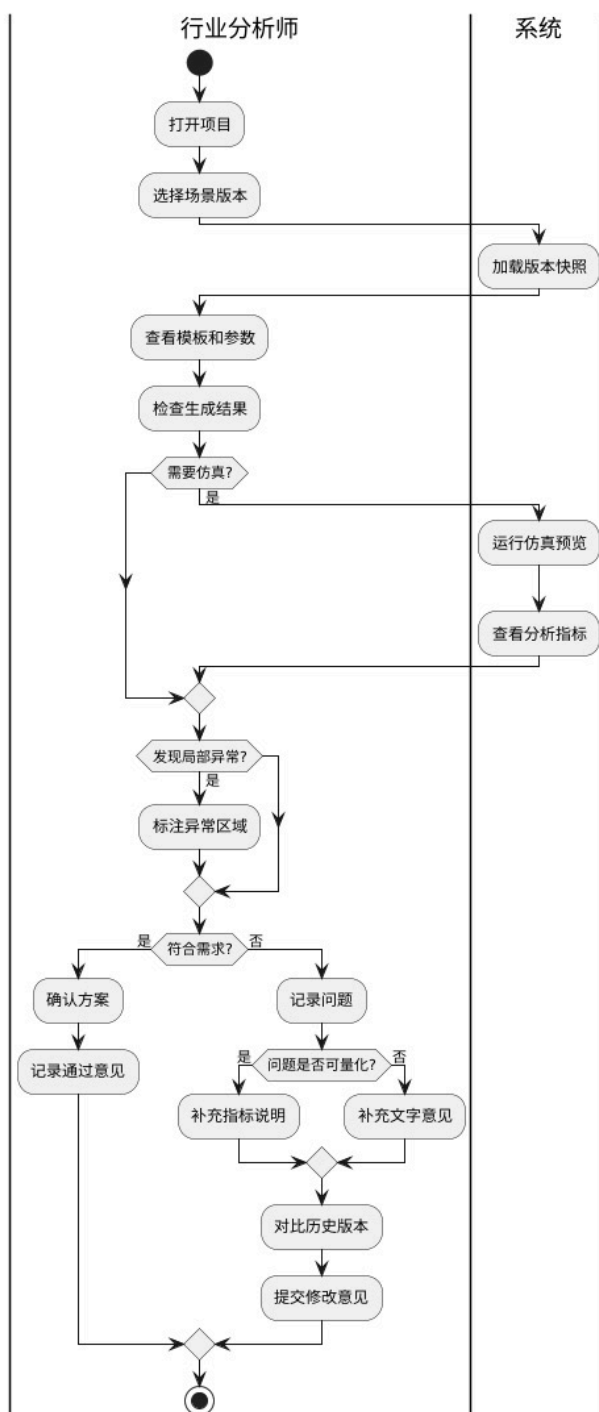


图 8: 行业分析师核对生成结果活动图

我们在图 8 中采用行业分析师和系统两条泳道，用于描述行业分析师对生成结果的核对流程。行业分析师首先打开项目并选择场景版本，系统随后执行加载版本快照，这也说明分析师查看的不是临时预览结果，而是之前已经被保存的某个场景版本。之后分析师依次查看模板和参数并检查生成结果，即核对该版本采用的模板规则、道路与建筑参数、绿化配置以及最终场景效果是否符合业务目标。

第一个判断节点是“需要仿真?”。如果需要进一步验证的话，系统泳道会执行运行仿真预览和查看分析指标，将人流、车流、光照或生态指标反馈给分析师；如果不需要仿真，则流程直接进入后续判断。接着，分析师通过“发现局部异常?”判断是否存在局部问题。如果发现异常的话，则需要先标注异常区域；如果没有异常，则跳过该步骤。随后流程进入“符合需求?”分支判断，若符合需求，则确认方案并记录通过意见；若不符合需求，则进入记录问题。

我们在这个流程里不仅安排了分析师查看结果的顺序，还区分了定量指标问题和定性修改意见，使核对结论能够回写到项目版本中，形成后续修改和方案比较的依据。例如，在不符合需求的分支中还设置了对于“问题是否可量化?”的判断。若问题可以用指标描述，分析师则补充指标说明；若问题更偏主观或展示效果，则补充文字意见，两类意见最终汇合后进入对比历史版本和提交修改意见。

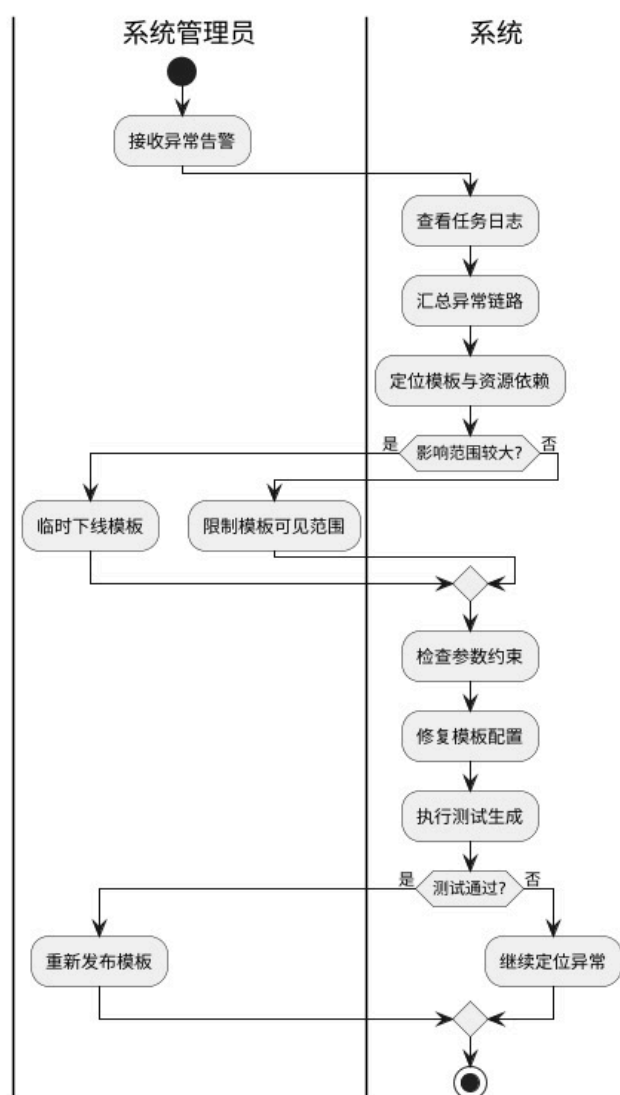


图 9: 系统管理员处理模板异常活动图

我们在图 9 中描述了系统管理员收到模板异常告警后的处理流程。管理员泳道从接收异常告警开始，系统泳道随后依次执行查看任务日志、汇总异常链路和定位模板与资源依赖，这三个活动说明模板异常排查并不是直接修改模板文件，而是先根据失败任务日志定位问题模板、依赖资源、参数约束和插件调用位置。

接着，我们通过“影响范围较大？”去判断异常的治理强度。如果影响范围较大，管理员需要执行临时下线模板，防止该模板继续被新任务使用；如果影响范围较小，则执行限制模板可见范围，只对受影响用户或项目隐藏模板。随后两个分支汇合到系统侧的治理流程，继续检查参数约束、修复模板配置和执行测试生成。这里的测试生成用于验证修复后的模板是否能够在真实资源和插件环境中正常运行，而不是只检查配置文本是否正确。

最后通过“测试通过？”判断是否结束异常处理。若测试通过，管理员执行重新发布模板，模板重新进入可用状态；若测试不通过，流程进入继续定位异常，返回排查阶段。这个流程的重点在于说明模板异常处理需要包含影响范围控制、配置修复、测试验证和重新发布四个环节，避免一个存在问题的道路、建筑或绿化模板持续影响多个城市生成任务。

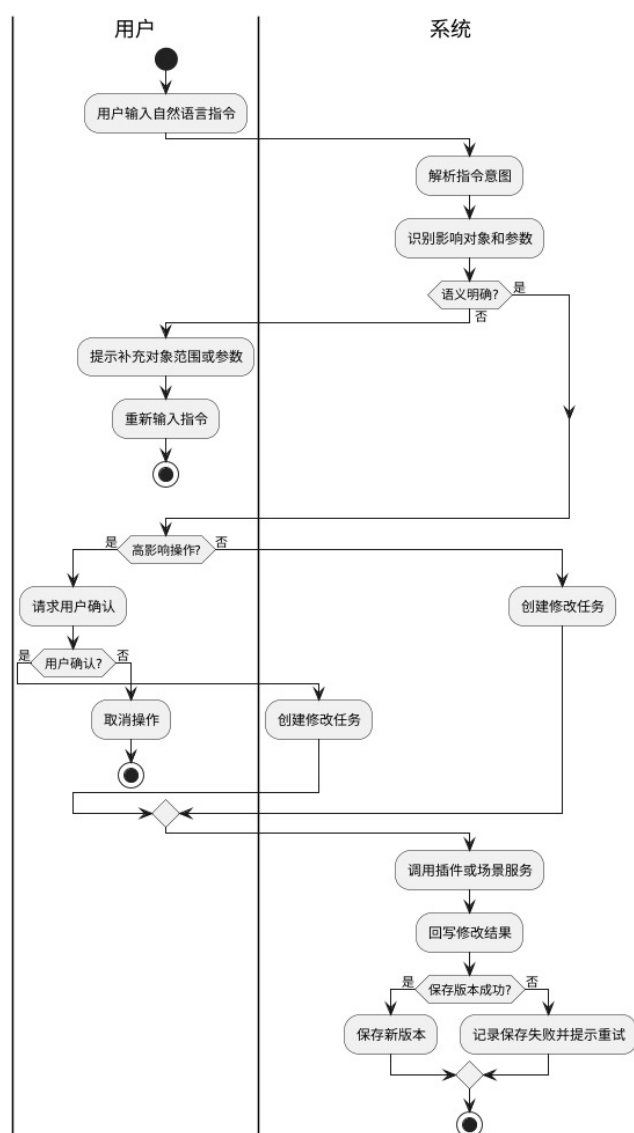


图 10: 自然语言修改场景活动图

我们用图 10 中的用户和系统两条泳道描述自然语言修改场景的执行过程。用户首先输入自然语言指令，系统接收后执行解析指令意图和识别影响对象和参数。这两个活动分别对应语义理解和对象定位，例如从“把主干道两侧的建筑再高一些”中识别出操作对象是主干道两侧建筑，操作类型是高度调整，参数是增高幅度或目标高度。

第一个判断节点是“语义明确?”。如果语义不明确，流程返回用户侧的提示补充对象范围或参数，用户重新输入指令后流程终止本次尝试，表示系统不会在范围、对象或参数不完整的情况下继续执行。如果语义明确，则进入“高影响操作?”判断。若该操作会大范围改变场景，例如批量修改建筑群、道路网络或绿化层，系统需要先请求用户确认，用户确认后才会在系统侧创建修改任务，若用户不确认，则执行取消操作并结束流程。

对于非高影响操作，系统可以直接创建修改任务。随后所有有效修改请求汇合，系统调用插件或场景服务，将结构化修改任务交给 Blender 插件或内部场景服务执行。执行完成后系统回写修改结果，再通过“保存版本成功?”判断结果是否被可靠沉淀。保存成功时进入保存新版本，保存失败时进入记录保存失败并提示重试。因此，我们在这个流程中不仅保留自然语言交互的便捷性，也明确了语义补全、风险确认、任务创建、结果回写和版本保存之间的控制顺序。

3.5 活动流程说明

四张活动图覆盖了系统的主要业务路径。场景建模师流程强调生产效率，行业分析师流程强调结果核对，系统管理员流程强调治理和风险控制，自然语言流程强调智能交互和操作确认。结合这些流程可以看出，系统并不是若干孤立功能的拼接，而是围绕场景生成任务组织起来的一条完整业务链。

在流程设计中，人工确认和人工精修环节都被保留下来。原因在于城市三维场景不仅要求生成速度，还要求空间关系合理、视觉风格一致和局部细节可控。自动生成可以快速搭建道路、建筑和环境框架，但局部边界、景观风格和展示角度往往仍需要专业用户判断，因此活动图中没有把自动生成作为流程终点，而是把它作为初始成果形成的阶段，后续仍允许用户继续编辑、保存版本和导出结果。

异常分支同样是活动图的重要组成部分。资源缺失时系统应引导用户补充资源，而不是直接终止任务；参数非法时系统应指出具体字段和范围，而不是只给出笼统错误；插件失败时系统应明确给出失败原因，并把用户引导回参数调整或模板切换阶段。通过这些流程约束，系统才能从演示型工具提升为具有工程可用性的生成平台。

4 类图

4.1 类图简介

类图 (Class Diagram) 用于描述面向对象系统中的类、属性、方法以及类之间的静态关系。类图强调系统中有哪些对象以及对象之间如何关联，是后续数据库设计、接口设计和代码实现的重要依据。类图中的关系通常包括关联、聚合、组合、依赖和继承，其中组合关系表示强拥有关系，聚合关系表示弱拥有关系，依赖关系则表示一个类在行为上需要使用另一个类。

智能城市生成系统具有较明显的数据对象特征。用户、项目、资源、模板、场景、生成任务、插件调用记录和版本记录都需要被系统长期管理。如果这些对象边界不清晰，实现阶段容易出现任务状态难以追踪、场景版本无法恢复、资源引用混乱等问题，因此需要通过类图对核心业务对象进行抽象。

类图通常可以从四个层次理解。第一层是类本身，包括类名、属性和方法；第二层是类之间的静态关系，包括关联、聚合、组合、依赖和继承；第三层是多重性，用于说明一个对象与另一个对象之间可能存在的一对一、一对多或多对多关系；第四层是辅助说明元素，例如枚举类型、关联名称和注释，

用于补充角色类型、任务状态、资源类别以及关键对象的设计意图。智能城市生成系统的类图重点关注业务对象之间的归属关系和任务执行关系，例如一个项目可以包含多个场景版本，一个生成任务会依赖一个模板并调用插件适配器，一个场景版本由某次生成或修改操作沉淀而来，而道路网、建筑群、植被层和交通对象又共同组成城市场景内部结构。

表 5: 类图关系符号说明

关系类型	含义	在本系统中的示例
关联	两个类之间存在业务联系，但生命周期相对独立	Template 与 GenerationTask 之间存在使用关系
聚合	整体与部分关系，部分可以脱离整体存在	Project 聚合 Asset，资源可以被多个项目复用
组合	强整体与部分关系，部分依附整体生命周期	Project 组合 Scene，场景通常依附项目存在
依赖	一个类在行为中临时使用另一个类	GenerationTask 依赖 PluginAdapter 执行插件调用
多重性	表示对象数量约束	一个 Project 对应多个 VersionRecord

4.2 类图设计思路

类图用于描述系统中的核心对象、对象属性、对象行为以及类之间的关系。智能城市生成系统的数据对象较多，因此我们在这里将类图分为两类：一是系统核心类图，用于展示用户、项目、场景、资源、模板等基础对象；二是任务、插件与版本管理局部类图，用于展示生成任务执行和结果沉淀过程中的对象关系。

4.3 核心类与职责划分

表 6: 核心类职责说明

类名	主要职责	关键说明
User	保存用户身份、角色和权限信息	不同角色决定用户可访问的项目、模板和后台功能
Project	组织项目级场景、资源、任务和版本	是大部分业务对象的归属边界
Scene	表示当前城市三维场景状态	包含道路、建筑、绿化、水体和动态对象等场景元素
Asset	表示底图、模型、材质等资源	需要记录资源类型、路径、校验状态和引用关系
Template	封装模板规则和默认参数	用于驱动场景自动生成并约束参数范围
GenerationTask	表示一次生成或修改任务	是连接用户操作、插件执行和结果保存的关键对象
VersionRecord	保存场景快照和版本说明	支持结果追踪、方案比较和异常回滚

系统核心类包括用户类、项目类、场景类、资源类、模板类、生成任务类、插件适配器类、仿真任务类、版本记录类和日志记录类。其中，用户类负责身份和权限信息；项目类负责组织场景、资源和版本；场景类承载城市空间对象；资源类表示底图、模型、材质和贴图；模板类封装生成规则和默认参数；生成任务类表示一次异步生成过程；插件适配器负责屏蔽不同 Blender 插件的调用差异；版本

记录类和日志记录类用于追踪系统状态。为了让类图更贴近城市生成业务，我们把场景内部继续细分为道路网、建筑群、植被层、水体系和交通对象等结构，它们共同决定场景的空间形态和动态表现。

4.4 类图构建

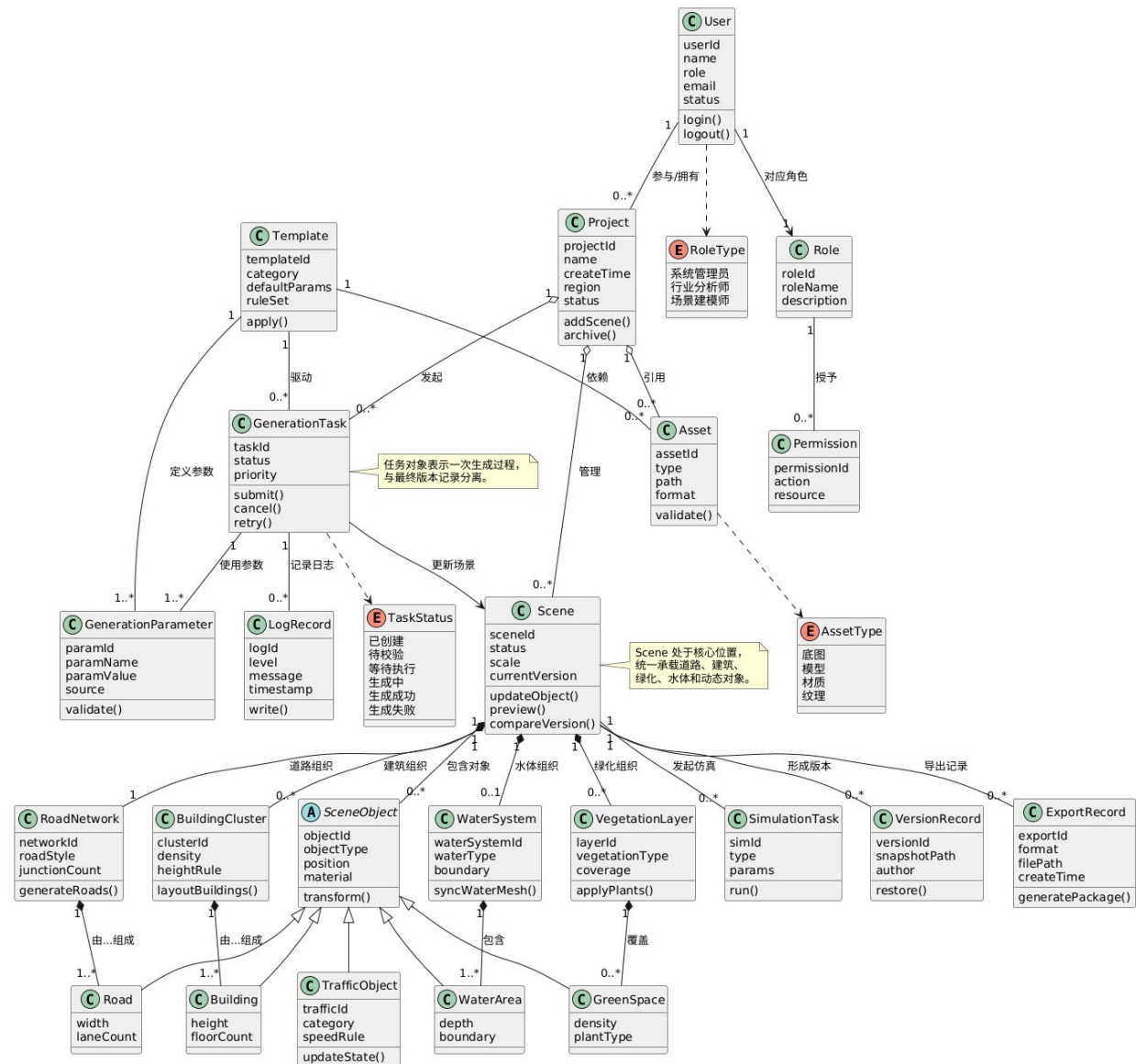


图 11: 智能城市生成系统核心类图

我们把图 11 设计为系统核心类图，上半部分放置用户、项目、资源、模板和任务等管理对象，下半部分放置城市场景内部的空间对象。用户类 User 包含 `userId`、`name`、`role`、`email`、`status` 等属性，并提供 `login()` 和 `logout()` 方法；它与 Project 之间存在参与/拥有关系，表示一个用户可以参与多个项目。用户还通过 Role 获得多个 Permission，并由枚举 RoleType 限定为系统管理员、行业分析师或场景建模师，从而为前文用例图中的权限边界提供对象基础。

我们把 Project 放在项目组织中心，属性包括 `projectId`、`name`、`createTime`、`region` 和 `status`，方法包括 `addScene()` 与 `archive()`。它与 Asset、GenerationTask、Scene 等对象相连，说明资源、生成任务和场景结果都需要归属到具体项目。资源类 Asset 记录 `assetId`、`type`、`path`、`format`，并

通过 `validate()` 校验资源有效性；其类型由 `AssetType` 枚举限制为底图、模型、材质和纹理。模板类 `Template` 包含 `defaultParams` 和 `ruleSet`，通过驱动关系关联 `GenerationTask`，表示模板规则会被生成任务读取并转化为具体生成参数。

在场景对象部分，`Scene` 是中心类，包含 `sceneId`、`status`、`scale`、`currentVersion`，并提供 `updateObject()`、`preview()` 和 `compareVersion()` 等方法。它以组合方式包含道路组织、建筑组织、水体组织、绿化组织和动态对象：`RoadNetwork` 由多个 `Road` 组成，用于描述道路风格、交叉口和车道数；`BuildingCluster` 由多个 `Building` 组成，用于描述建筑密度、高度规则和楼层数；`VegetationLayer` 覆盖多个 `GreenSpace`，用于记录植被类型、覆盖率和植物类别；`WaterSystem` 包含多个 `WaterArea`，用于描述水体类型、边界和深度；`TrafficObject` 继承 `SceneObject`，用于表达车辆、信号灯或站点等动态设施。抽象类 `SceneObject` 统一了 `objectId`、`objectType`、`position`、`material` 和 `transform()`，说明不同城市场景对象虽然形态不同，但都具备位置、材质和变换等共同属性。

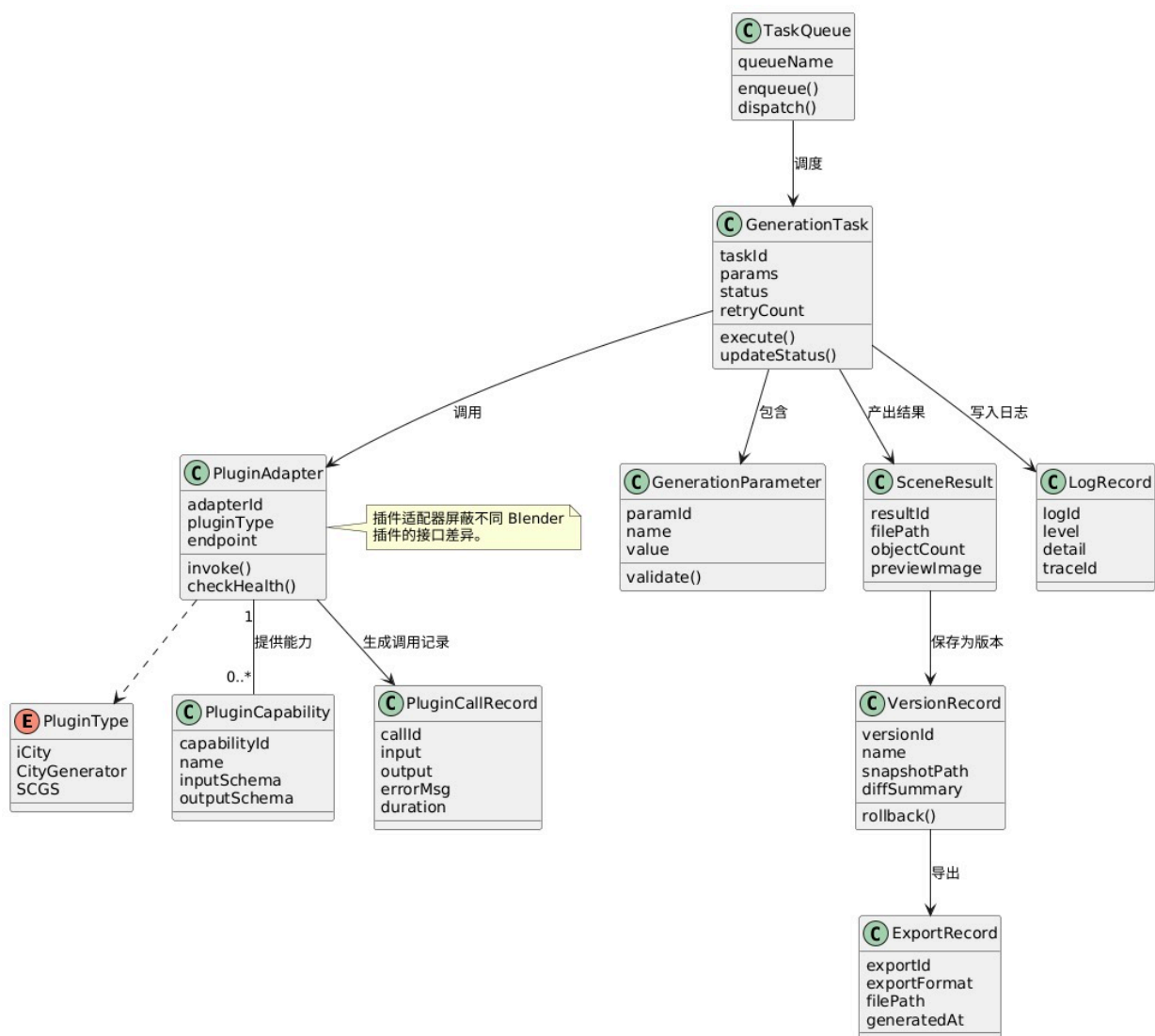


图 12: 任务、插件与版本管理局部类图

我们在图 12 中围绕任务执行—插件调用—结果沉淀进一步绘制局部类图。顶部的 `TaskQueue` 通过 `enqueue()` 和 `dispatch()` 调度 `GenerationTask`，表示生成任务通常以异步队列方式执行，而不是由前端请求直接阻塞等待。 `GenerationTask` 包含 `taskId`、`params`、`status` 和 `retryCount`，并提

供 `execute()` 与 `updateStatus()` 方法, 用于记录一次生成或修改任务的执行过程。

任务执行时会包含多个 `GenerationParameter`, 每个参数记录 `paramId`、`name` 和 `value`, 并通过 `validate()` 检查合法性。生成任务调用 `PluginAdapter`, 而适配器包含 `adapterId`、`pluginType`、`endpoint`, 并提供 `invoke()` 和 `checkHealth()` 方法。其中 `PluginAdapter` 一侧连接 `PluginCapability` 与 `PluginType`, 说明适配器既要知道插件属于 `iCity`、`CityGenerator` 或 `SCGS` 等哪一类, 也要记录该插件提供的输入输出模式; 另一侧连接 `PluginCallRecord`, 用于保存每次插件调用的 `input`、`output`、`errorMsg` 和 `duration`。这能支撑后续排错、审计和性能分析。

右侧的 `SceneResult`、`VersionRecord`、`ExportRecord` 和 `LogRecord` 体现了结果管理。插件返回后先形成 `SceneResult`, 其中包含结果文件路径、对象数量和预览信息; 该结果进一步保存为版本, 生成 `VersionRecord`, 记录 `versionId`、`name`、`snapshotPath` 和差异摘要, 并支持 `rollback()` 回滚; 需要交付时再由 `ExportRecord` 记录导出格式、文件路径和生成时间。与此同时, `GenerationTask` 还会写入日志到 `LogRecord`。因此, 我们在这张图中把一次生成任务从参数校验、插件适配、调用记录、结果生成到版本保存和导出记录的全过程都对象化了。

4.5 类间关系说明

类间关系方面, 项目与场景、资源、任务、版本之间多为组合或聚合关系, 说明这些对象通常依附于具体项目存在。模板与资源之间为关联关系, 因为模板会引用资源但不拥有资源。生成任务依赖模板、资源和插件适配器, 任务执行结果最终形成新的场景版本。日志记录与任务、用户、插件调用之间保持弱关联, 用于后续检索和问题定位。

类图中还需要注意任务和结果的分离。生成任务表示一次执行过程, 它关注参数、状态、执行时间、失败原因和重试次数; 场景版本表示执行后的成果, 它关注快照路径、版本说明、作者和是否可回滚。如果把任务和版本混为一个对象, 系统在处理失败任务、历史版本比较和成果导出时会变得混乱, 因此将 `GenerationTask` 和 `VersionRecord` 分开建模, 并通过 `SceneResult` 建立两者之间的过渡关系。

此外, 类图中采用了分组层和对象层两级表示方式。`RoadNetwork`、`BuildingCluster`、`VegetationLayer` 和 `WaterSystem` 表示城市场景中的领域分组结构, 用来体现道路网、建筑群、植被层和水体系统的整体组织关系; `Road`、`Building`、`GreenSpace` 和 `WaterArea` 则表示这些分组内部更细粒度的具体对象。这样处理之后, 既能在类图中保留城市生成系统的宏观层次, 也能说明底层场景对象如何被统一抽象。通过抽象父类统一位置、材质和变换方法, 可以避免每增加一种场景对象都重新设计基础操作。类图中的箭头、多重性和关联名称都直接对应业务约束, 其中 `Scene` 处于场景组织的中心, `GenerationTask` 处于生成调度的中心, 这两个对象共同把城市内容与生成过程连接起来。

5 顺序图

5.1 顺序图简介

顺序图 (Sequence Diagram) 是 UML 交互图的一种, 用于描述参与对象之间按照时间顺序发生的消息传递过程。顺序图通常包含参与者、对象生命线、消息箭头和返回消息, 它能够清楚展示一次业务请求从用户发起到系统响应之间经历了哪些模块, 以及各模块调用的先后关系。

在智能城市生成系统中, 顺序图很适合描述异步任务和外部服务调用。模板生成、自然语言修改、插件调用和结果导出都不是简单的本地计算, 而是涉及前端、后端服务、资源服务、插件适配器、外

部插件环境和版本服务之间的连续协作，把这些调用顺序画清楚之后，实现阶段就不容易遗漏资源校验、状态更新和异常记录等关键步骤。

顺序图中最重要的元素是生命线和消息。生命线表示某个对象在交互过程中的存在时间，消息箭头表示对象之间的调用方向。同步消息通常意味着调用方需要等待返回结果，异步消息则适合任务队列、插件执行和后台导出等耗时操作。条件片段用于表示分支判断，可选片段用于表示仅在特定条件下发生的附加步骤，循环片段用于表示需要重复执行直到满足条件的交互过程，例如登录失败时返回错误信息，插件执行失败时记录日志并进入重试流程，自然语言语义不完整时继续补充参数后重新解析。

表 7: 顺序图常用元素说明

元素	图中含义	在本系统中的示例
参与者	发起交互的外部角色	场景建模师提交生成任务，行业分析师查看仿真结果
生命线	参与交互的对象或服务实例	前端工作台、任务服务、插件适配器、版本服务
同步消息	需要等待返回的调用	查询用户信息、读取模板定义、保存版本
异步消息	可后台执行的调用	插件执行、导出文件、仿真计算
条件片段	不同条件下执行不同消息序列	登录成功或失败、插件成功或失败、是否需要用户确认
可选片段	仅在满足特定条件时追加执行的消息序列	登录成功后加载角色入口、任务完成后发送通知
循环片段	同一组消息在条件满足前重复执行	自然语言指令语义不完整时反复补充参数并重新解析

5.2 顺序图设计思路

顺序图用于描述参与对象之间按照时间顺序发生的消息交互。我们这里选择四个典型交互场景进行建模：用户登录与权限校验、模板驱动场景生成、自然语言驱动场景修改、结果导出与版本保存，这四个场景覆盖了系统从入口验证到核心生成再到结果沉淀的主要调用链。

5.3 典型交互场景选择

用户登录与权限校验是所有业务操作的前置条件；模板驱动场景生成是系统最基本的生产流程；自然语言驱动场景修改体现系统智能化特征；结果导出与版本保存则保证生成成果能够被追踪、复用和交付。

表 8: 顺序图场景与关键消息

场景	关键消息	设计重点
登录与权限校验	login、queryUser、returnToken	认证结果必须同时返回角色和可访问功能范围
模板驱动生成	createTask、checkAssets、invokePlugin、saveScene	任务执行前必须完成资源和参数校验
自然语言修改	parse、confirm、createModifyTask、updateScene	高影响操作需要用户确认后执行
导出与版本保存	saveCurrentVersion、export、writeFile、recordExport	导出前保存版本，导出后记录日志

5.4 顺序图构建

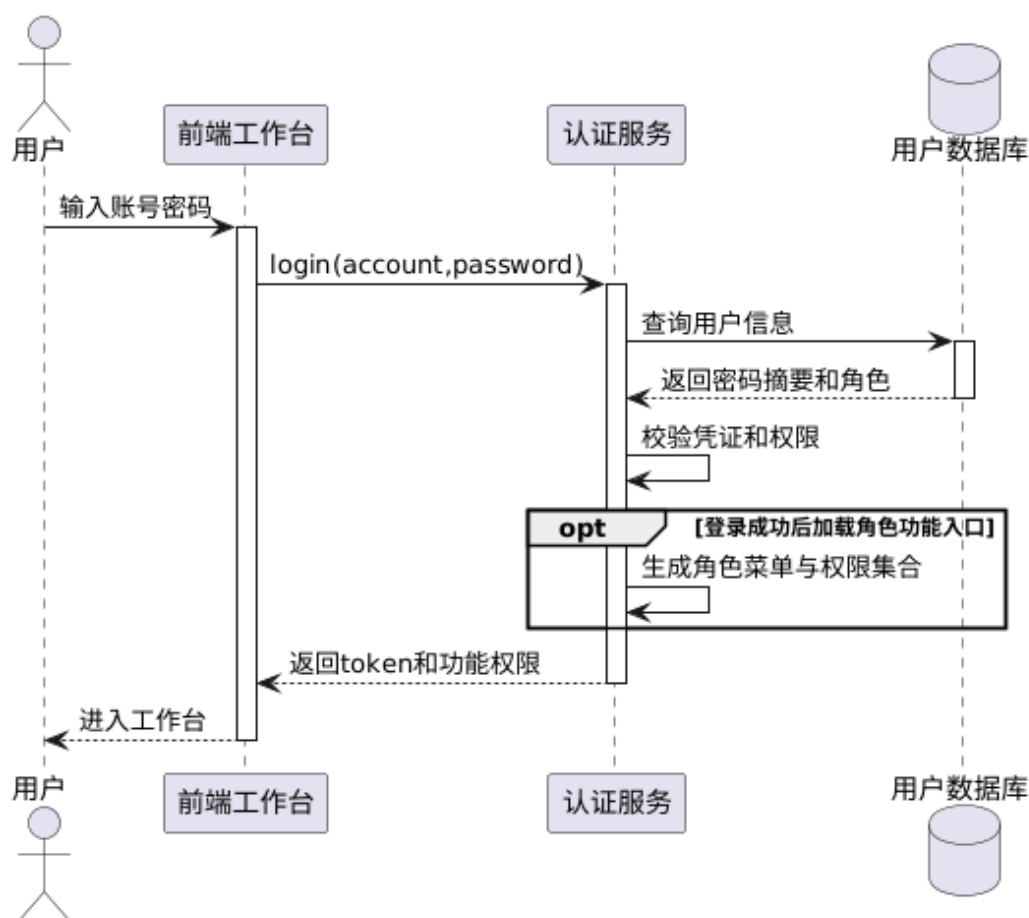


图 13: 用户登录与权限校验顺序图

我们用图 13 描述用户登录与权限校验的消息顺序。用户首先在前端工作台输入账号和密码，前端随后向认证服务发送 `login(account,password)` 消息。认证服务接着访问用户数据库，查询用户基本信息，并从数据库返回密码摘要和角色信息。认证服务在本地完成校验凭证和权限，即同时判断密码是否正确、账号是否可用以及角色是否拥有进入系统的权限。

其中的 `opt` 片段标注为“登录成功后加载角色功能入口”。这表示该片段只在认证通过时执行：认证服务根据用户角色生成角色菜单与权限集合，再把 `token` 和功能权限返回给前端工作台。前端获得这些信息后，用户才正式进入工作台。如果认证失败，该可选片段不会执行，前端也不会加载场景建模、结果分析或后台管理入口。由此可见，我们在这部分顺序设计中不仅说明登录消息从用户到数据库的传递过程，还说明角色菜单是在认证成功后一次性生成的，避免后续页面再重复进行分散的权限判断。

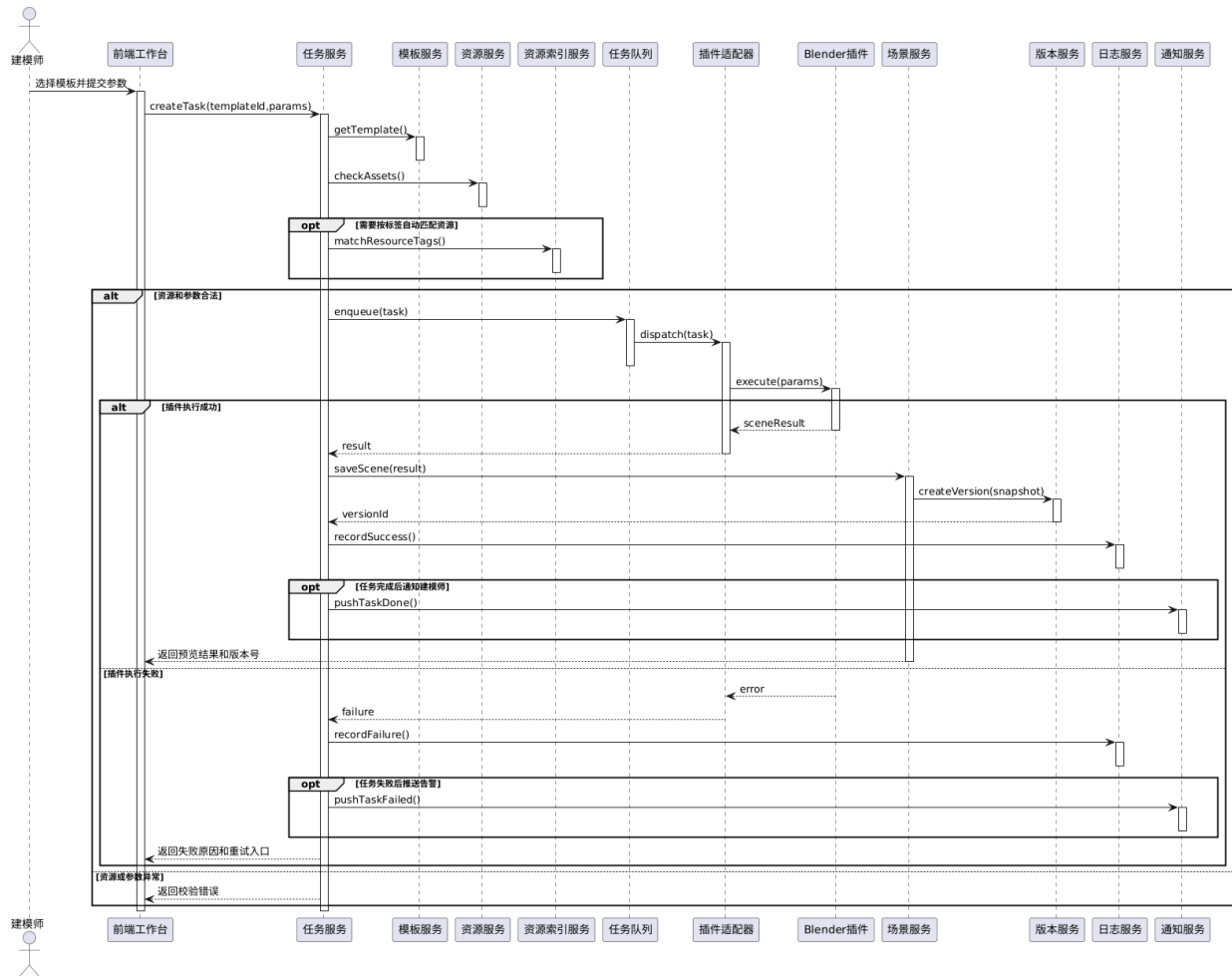


图 14: 模板驱动场景生成顺序图

我们在图 14 中按时间顺序安排模板驱动场景生成的完整调用链。建模师在前端工作台选择模板并提交参数后，前端向任务服务发送 `createTask(templateId,params)`。任务服务首先向模板服务发送 `getTemplate()` 读取模板规则，然后向资源服务发送 `checkAssets()` 检查底图、模型、材质和纹理是否满足模板依赖。我们还设置了一个 `opt` 片段“需要按标签自动匹配资源”，其中通过资源索引服务执行 `matchResourceTags()`，表示当项目资源较多时，系统可以根据标签自动匹配道路、建筑或绿化资源。

主干流程进入 `alt` 片段后分为资源和参数合法、插件执行成功、插件执行失败、资源或参数异常等路径。在“资源和参数合法”路径下，任务服务把任务写入任务队列 `enqueue(task)`，任务队列再将任务分发给插件适配器 `dispatch(task)`。插件适配器向 Blender 插件发送 `execute(params)`，插件执行后返回 `sceneResult`。如果执行成功，任务服务获得 `result` 后调用场景服务 `saveScene(result)` 保存场景结果，并由场景服务调用版本服务 `createVersion(snapshot)` 生成版本编号。随后任务服务通过日志服务 `recordSuccess()` 记录成功日志，并可选地通过通知服务 `pushTaskDone()` 通知建模师任务完成。

如果插件执行失败，Blender 插件向插件适配器返回 `error`，任务服务得到 `failure` 后调用日志服务 `recordFailure()` 记录失败原因，并可选地通过通知服务 `pushTaskFailed()` 发送告警。若资源或参数在前置校验阶段异常，则流程不进入插件调用，而是直接向前端返回校验错误。这样，我们就成功把模板读取、资源检查、任务入队、插件执行、场景保存、版本创建、日志记录和用户通知按时间

顺序串联了起来，从而可以明确说明哪些消息属于正常生成路径，哪些消息属于失败处理路径。

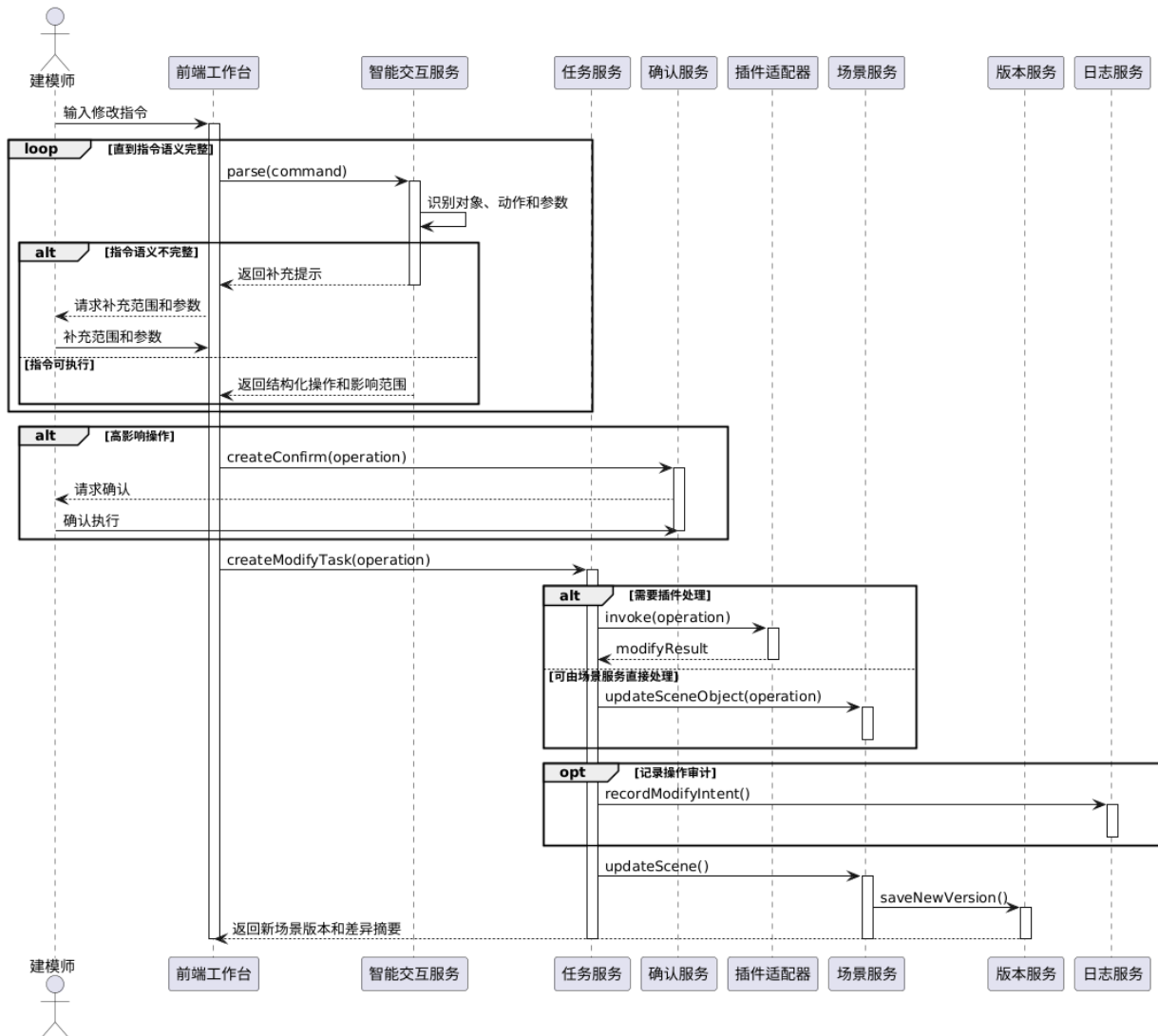


图 15: 自然语言驱动场景修改顺序图

我们用图 15 描述自然语言修改场景时各服务之间的交互顺序。建模师输入修改指令后，前端工作台向智能交互服务发送 `parse(command)`，其中最外层 `loop` 片段表示系统会反复解析，直到指令语义完整为止。智能交互服务首先识别对象、动作和参数，若进入“指令语义不完整”分支，则向前端返回补充提示，用户补充范围和参数后再次解析，若进入“指令可执行”分支，则返回结构化操作和影响范围。

随后我们设置第二个 `alt` 片段，用于处理高影响操作，当前端收到结构化操作后，如果系统判定该操作影响范围较大，前端会向确认服务发送 `createConfirm(operation)`，确认服务请求用户确认，用户确认后才继续执行，确认完成后，前端向任务服务发送 `createModifyTask(operation)` 创建修改任务。任务服务在执行阶段又通过 `alt` 片段区分两种处理方式：如果需要插件处理，则向插件适配器发送 `invoke(operation)`，由插件返回 `modifyResult`；如果可以由内部场景服务直接处理，则调用 `updateSceneObject(operation)` 更新场景对象。

我们还保留了 `opt` 片段“记录操作审计”，任务服务通过日志服务执行 `recordModifyIntent()`，用于记录本次自然语言修改的原始意图、解析结果和影响范围。场景更新完成后，场景服务执行 `updateScene()`，

再由版本服务 `saveNewVersion()` 保存新版本，最后前端收到返回新场景版本和差异摘要。这种顺序设计说明自然语言修改不是一句话直接覆盖场景，而是经过语义补全、风险确认、任务创建、插件或场景服务执行、审计记录和版本保存后才生效。

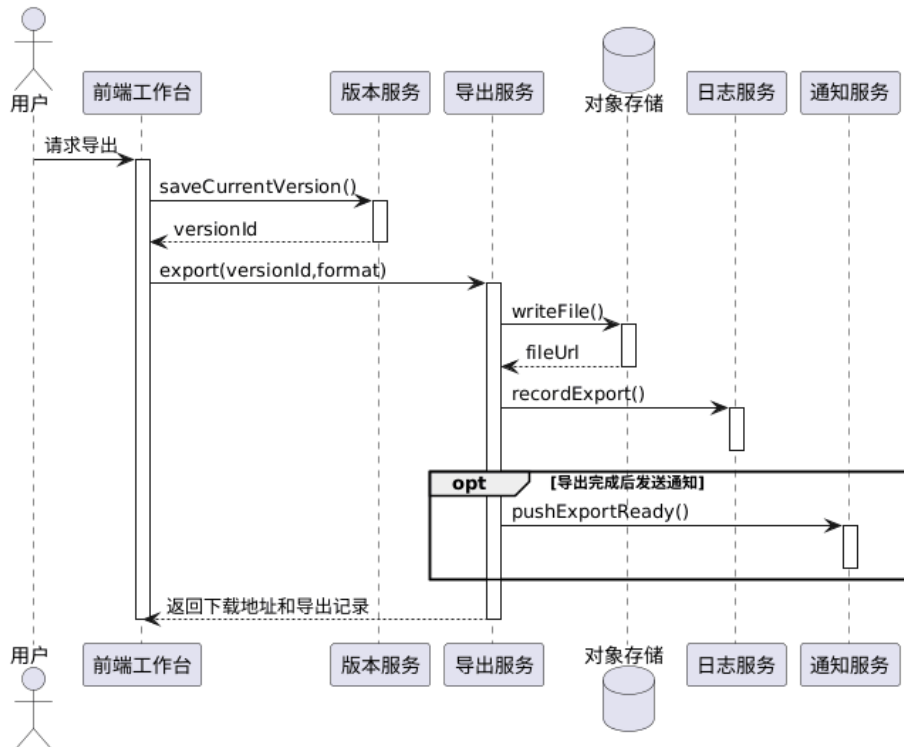


图 16: 结果导出与版本保存顺序图

我们用图 16 描述结果导出与版本保存的消息顺序。用户在前端工作台发起请求导出后，前端首先调用版本服务 `saveCurrentVersion()`，版本服务返回 `versionId`，这一步说明导出前必须先把当前场景固化为一个明确版本，后续导出文件才能与具体版本对应。

获得版本编号后，前端向导出服务发送 `export(versionId,format)`，其中 `format` 表示用户选择的导出格式。导出服务将生成文件写入对象存储 `writeFile()`，对象存储返回 `fileUrl`，随后导出服务调用日志服务 `recordExport()`，记录导出版本、文件路径、格式和操作时间，其中的 `opt` 片段表示“导出完成后发送通知”，系统可以调用通知服务 `pushExportReady()` 告知用户下载已就绪。

最后前端向用户返回下载地址和导出记录。这一顺序设计的核心是把版本保存和文件导出分成两个阶段：版本服务负责证明导出对象来自哪个场景状态，导出服务负责生成可交付文件，对象存储负责保存大体积模型和资源包，日志服务负责追踪导出行为，这样即使同一场景被多次导出为不同格式，也可以追溯每个文件对应的版本来源。

5.5 消息传递过程说明

这四张顺序图体现了系统的分层调用结构：前端工作台负责收集用户输入，业务服务负责校验和调度，插件适配器负责调用外部执行环境，场景服务和版本服务负责保存结果。通过这种结构，系统可以在插件失败或外部服务不可用时保持主流程可控，避免错误直接破坏已有场景。

顺序图也说明了系统中同步调用和异步调用的边界。登录、模板查询和资源校验通常应采用同步调用，因为用户需要立即得到反馈；插件执行、场景导出和仿真计算则更适合异步处理，因为这些任

务耗时较长，如果全部阻塞在前端请求中，会导致页面卡顿和任务状态不可追踪，因此在模板驱动场景生成顺序图中，任务服务先创建任务，再交由任务队列和插件适配器处理。

自然语言修改场景的顺序图体现了一个重要设计原则：智能解析结果不能直接覆盖场景。特别是大语言模型可能存在理解偏差，用户输入也可能含糊，例如“把路边绿化调密一点”并没有明确作用范围和密度值。因此系统需要先将指令转换为结构化操作，并在影响范围较大时请求用户确认，这样既保留自然语言交互的效率，又控制了自动化带来的风险。在我们设计的顺序图中的每一条消息都可以对应到城市生成过程中的一个实际动作：读取模板规则、校验资源依赖、调度插件执行、保存场景版本、记录导出结果，这样的链路说明比单纯写调用插件生成场景更接近系统真实运行方式。

6 协作图

6.1 协作图简介

协作图（Communication Diagram）也称通信图，是 UML 中用于表达对象之间协作结构的一类交互图，它与顺序图都可以描述对象之间的消息传递，但关注点不同：顺序图强调时间先后，协作图强调对象之间的连接关系以及消息沿这些连接关系如何传递。因此，在分析一个功能由哪些对象共同完成时，协作图比顺序图更能体现系统结构。

智能城市生成系统中的许多任务都需要多个对象协同完成。以模板驱动生成场景为例，前端工作台、模板库、资源库、任务服务、插件适配器、场景服务和版本服务都会参与，而协作图能够把这些对象之间的连接关系和消息编号放在同一张图中，更适合观察它们如何共同完成一次任务。

协作图中的编号非常重要，它表示消息发生的先后顺序，与顺序图从上到下阅读不同，协作图更像一张对象网络图，需要结合消息编号理解流程。对于模块较多的系统，协作图能够帮助读者观察哪些对象处于协作中心、哪些对象只承担辅助角色，例如在模板生成过程中，任务服务是协作中心；在插件调用过程中，插件适配器是协作中心；在仿真分析过程中，仿真服务和分析视图之间的关系更加突出。

表 9: 协作图常用元素说明

元素	图中含义	在本系统中的示例
对象	参与某次协作的实例或模块	前端工作台、任务服务、模板库、插件适配器
链接	对象之间能够通信的连接关系	前端工作台与任务服务之间的请求链路
编号消息	按顺序标注的调用行为	1 查询模板, 2 校验资源, 3 调用插件
返回信息	对象处理后的结果反馈	返回场景预览、返回版本编号、返回分析指标

6.2 协作图设计思路

协作图也称通信图，用于强调对象之间的连接关系和消息传递顺序。与顺序图相比，协作图更适合展示某一功能场景中对象之间的组织结构。这里选择模板驱动场景生成、插件调用与结果回写、仿真预览与分析核对三个场景构建协作图。

6.3 协作对象分析

协作图中的对象主要包括前端工作台、项目服务、模板库、资源库、任务服务、插件适配器、Blender 插件、场景服务、版本服务、仿真服务和分析视图，不同场景下对象组合不同，但整体都围绕项目和场景展开。

表 10: 协作图中的主要对象

对象	所属模块	协作职责
前端工作台	前端交互层	收集用户操作并展示任务结果
任务服务	后端业务层	组织任务参数、调度插件执行、更新任务状态
模板库与资源库	数据与资产层	提供生成规则、默认参数和资源依赖
插件适配器	插件协同层	屏蔽具体插件差异，统一调用和结果回写
场景服务与版本服务	成果管理层	保存场景结果并生成可追踪版本

6.4 协作图构建

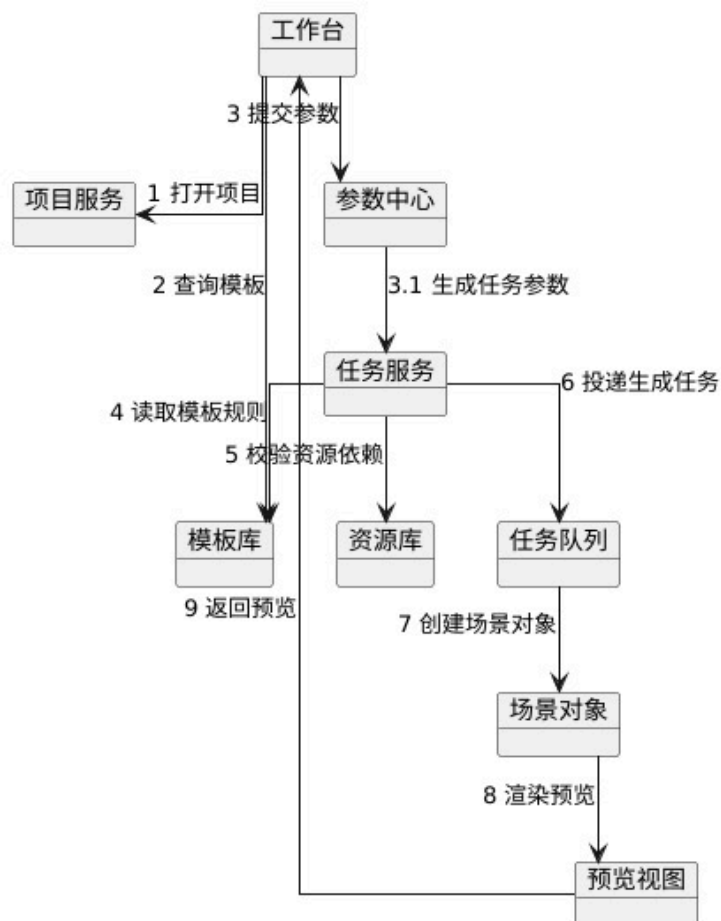


图 17: 模板驱动场景生成协作图

我们在图 17 中以对象网络的形式描述模板驱动生成时各对象的协作关系。消息 1 从工作台发往项目服务，表示用户打开项目；消息 2 查询模板，工作台从模板库获取可用模板；消息 3 是建模师在工作台提交参数；消息 3.1 表示参数中心根据用户输入和模板默认值生成任务参数。随后任务服务开始承担协作中心职责，消息 4 从任务服务到模板库读取模板规则，消息 5 到资源库校验资源依赖，说明任务创建前必须同时确认规则和资源。

在校验完成后，任务服务通过消息 6 将生成任务投递给任务队列，表示后续生成以异步任务方式执行。消息 7 指向场景对象，表示系统根据任务参数创建或更新内部场景对象；消息 8 从场景对象到预览视图，表示对当前生成结果进行渲染预览；消息 9 返回预览，最终把生成效果反馈到工作台。与顺序图相比，协作图不强调时间轴，而是强调任务服务、模板库、资源库、任务队列、场景对象和预览视图之间的连接关系，其中任务服务是连接规则、资源和结果对象的核心节点。

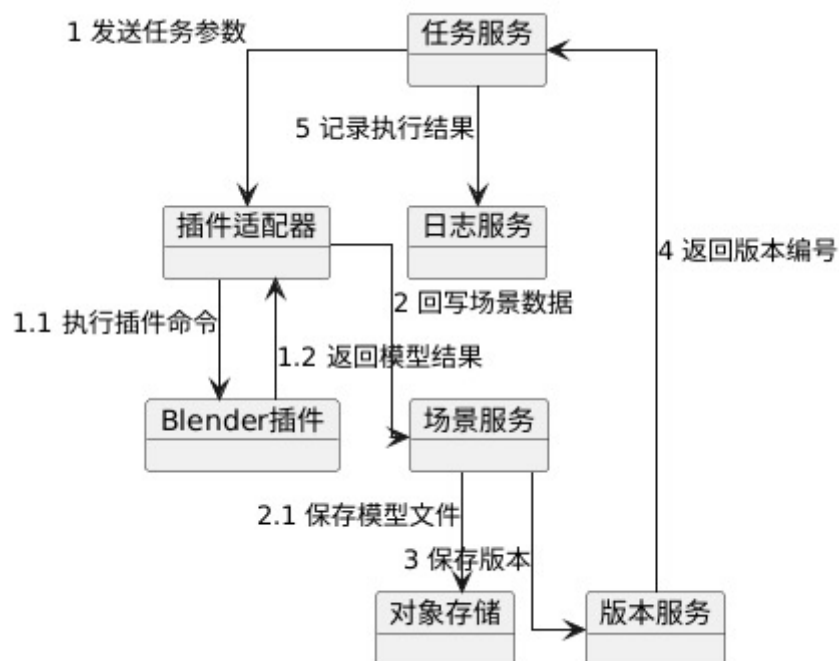


图 18: 插件调用与结果回写协作图

我们用图 18 表达插件调用与结果回写时的对象协作。消息 1 从任务服务发送任务参数到插件适配器，插件适配器并不直接保存结果，而是通过消息 1.1 向 Blender 插件执行插件命令。Blender 插件完成道路、建筑、绿化或水体生成后，通过消息 1.2 返回模型结果给插件适配器，这里插件适配器的作用是屏蔽具体 Blender 插件的命令格式，使主系统只面对统一的任务参数和返回结果。

结果返回后，插件适配器通过消息 2 将场景数据回写到场景服务；场景服务通过消息 2.1 将模型文件保存到对象存储，再通过消息 3 保存版本到版本服务。版本服务生成版本号后，通过消息 4 返回给任务服务，与此同时，消息 5 表示任务服务记录执行结果到日志服务。这说明插件生成结果不会停留在 Blender 环境中，而是经过场景服务、对象存储和版本服务纳入项目生命周期管理，最终由日志服务留下可审计记录。

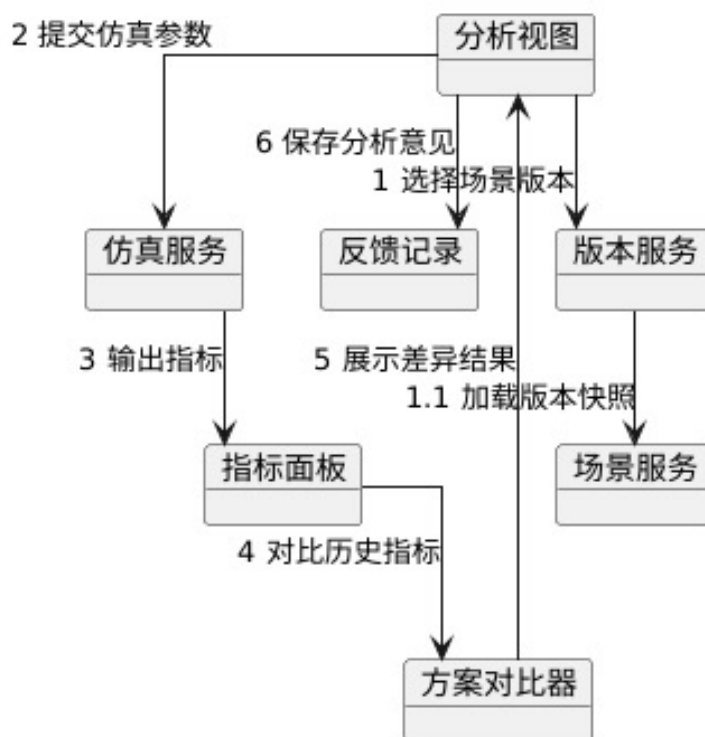


图 19: 仿真预览与分析核对协作图

我们在图 19 中描述行业分析师进行仿真预览和方案核对时的协作对象。消息 1 从分析视图发送到版本服务，用于选择场景版本；版本服务再通过消息 1.1 从场景服务加载版本快照，并把结果返回给分析视图。消息 2 表示分析视图提交仿真参数到仿真服务，仿真服务执行后通过消息 3 输出指标到指标面板。

分析师查看指标后，指标面板通过消息 4 对比历史指标，将当前版本与历史版本或目标阈值进行比较。随后分析视图调用方案对比器，消息 5 展示差异结果，用来呈现不同方案在道路通行、建筑密度、绿化覆盖或其他指标上的差异，最后，分析视图通过消息 6 将分析意见保存到反馈记录。这说明仿真结果不是一次性展示信息，而是会和版本快照、指标面板、方案对比器以及反馈记录共同组成可追踪的结果核对链路。

6.5 对象协作关系说明

协作图体现出系统对象之间的结构性依赖。任务服务处于调度中心位置，但不直接承担所有业务能力，模板库、资源库、插件适配器和仿真服务分别负责不同领域能力。这样既降低了模块耦合，也便于后续扩展新的插件、模板和仿真对象。

与顺序图相比，协作图更容易看出系统中的中心对象。例如在模板驱动场景生成协作图中，任务服务连接了模板库、资源库和场景对象，说明它承担业务编排职责；在插件调用与结果回写协作图中，插件适配器连接任务服务、插件执行环境和场景服务，说明它承担外部能力适配职责；在仿真预览与分析核对协作图中，分析视图、仿真服务和反馈记录形成闭环，说明行业分析师的意见最终也会成为项目数据的一部分。

这种协作关系有利于后续分工实现，前端部分可以重点完成前端工作台和分析视图，后端部分可以重点实现任务服务和版本服务，插件部分则围绕插件适配器提供统一接口。只要对象之间的消息约定保持稳定，不同模块就能够相对独立地迭代。

7 状态图

7.1 状态图简介

状态图（State Diagram）用于描述某个对象在生命周期中可能处于的状态，以及触发状态迁移的事件，它特别适合描述状态变化频繁、存在失败恢复或需要严格控制生命周期的对象。状态图通常包含初始状态、普通状态、终止状态和状态迁移条件。

在智能城市生成系统中，生成任务和场景版本都具有明确的生命周期。生成任务可能处于待校验、等待执行、生成中、生成成功、生成失败、已取消和已归档等状态；场景版本则可能经历草稿、已保存、待审核、已发布、已归档和已回滚等状态，通过状态图可以避免系统在实现中出现任务状态不一致、失败任务无法重试或版本覆盖不可恢复等问题。

状态图设计时需要区分状态和动作。状态表示对象在一段时间内保持的稳定情况，例如生成任务处于“生成中”；动作则表示状态变化时执行的行为，例如进入“生成中”状态时启动插件执行，进入“生成失败”状态时记录错误日志。状态迁移通常由事件触发，例如校验通过、插件返回结果、用户取消任务、系统管理员回滚版本等，而状态图的重点就是把这些触发条件和保护机制说清楚。

表 11: 状态图常用元素说明

元素	图中含义	在本系统中的示例
初始状态	对象生命周期开始点	生成任务创建后进入待校验状态
普通状态	对象在某阶段的稳定情况	等待执行、生成中、待审核、已发布
迁移	从一个状态切换到另一个状态	校验通过后从待校验进入等待执行
事件	触发迁移的外部或内部条件	插件返回成功、用户取消、系统管理员回滚
终止状态	对象生命周期结束点	任务归档、版本归档、任务取消

7.2 状态图设计思路

状态图用于描述某个对象在生命周期中的状态变化，智能城市生成系统中状态变化最明显的对象包括生成任务和场景版本。其中，生成任务需要经历创建、校验、执行、失败、重试和完成；而场景版本则需要经历草稿、已保存、待审核、已发布、已归档和已回滚等状态。

7.3 生成任务生命周期分析

生成任务是连接用户操作、资源校验、插件调用和结果保存的关键对象。任务创建后首先进行资源和参数校验，校验通过后进入等待执行状态。插件执行过程中，任务处于生成中状态；若执行成功，则生成场景结果并保存版本；若失败，则记录错误并允许用户重试或取消。

表 12: 生成任务关键状态说明

状态	含义	可执行操作
待校验	任务已经创建但尚未完成资源和参数检查	校验资源、校验参数、取消任务
等待执行	校验通过，任务等待插件执行节点处理	开始执行、取消任务
生成中	插件或场景服务正在执行生成逻辑	查看进度、记录日志
生成失败	任务执行过程中发生错误	查看原因、重试、取消
已归档	结果已经保存为场景版本	查看版本、导出成果、发起新任务

7.4 状态图构建

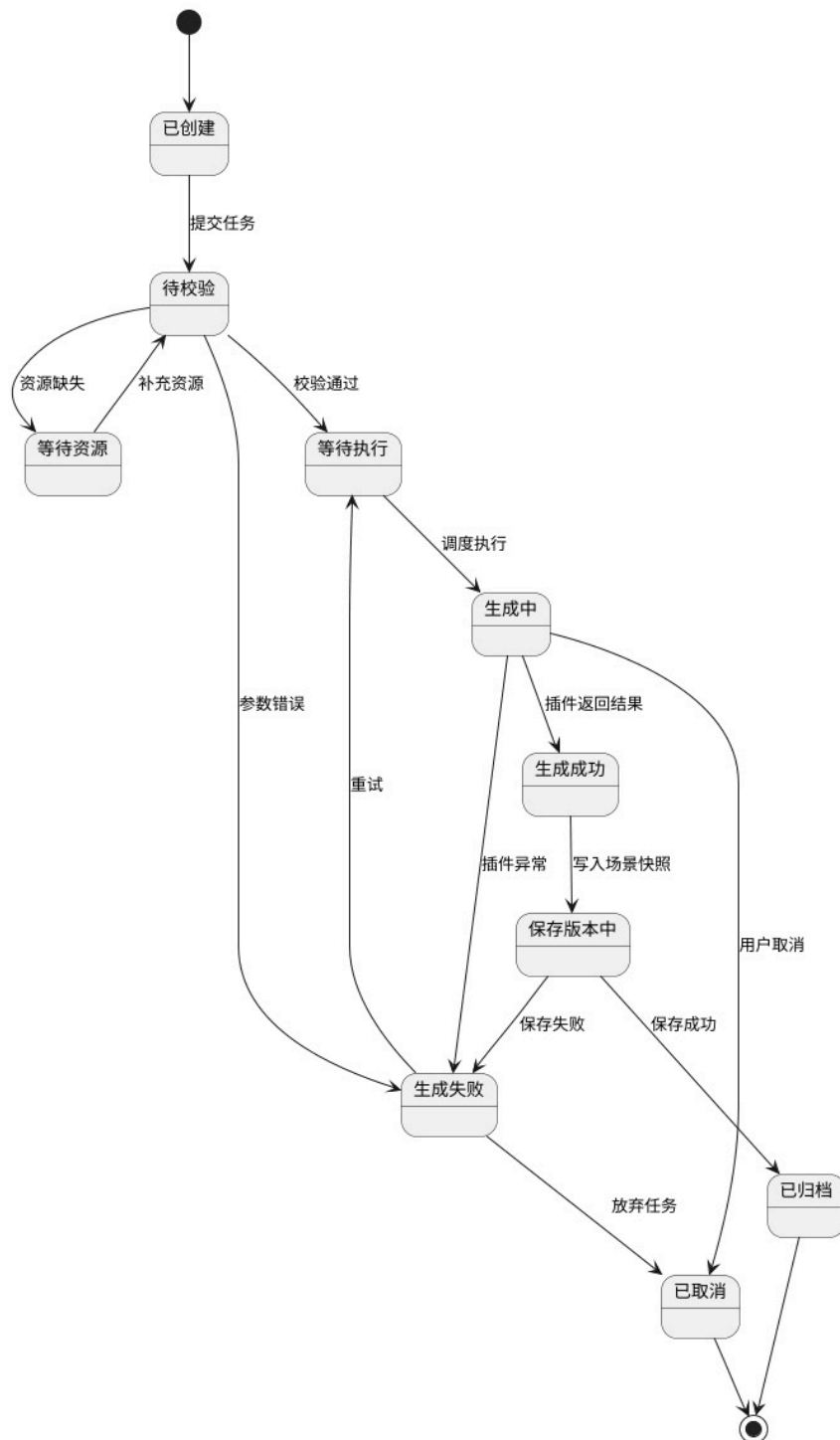


图 20: 生成任务状态图

我们用图 20 描述生成任务从创建到结束的状态变化。初始节点进入“已创建”，表示用户已经提交了生成意图但任务尚未校验；当发生提交任务事件后，任务进入“待校验”。在“待校验”状态下，系统检查资源完整性和参数合法性，如果资源缺失，任务迁移到“等待资源”；当用户补充资源后又回到“待校验”。如果参数错误，任务直接迁移到“生成失败”，说明参数校验失败不会进入插件执行阶段。

当校验通过后，任务进入“等待执行”，由任务队列等待调度。发生调度执行事件后，状态进入“生成中”。在“生成中”状态下，如果插件正常返回结果，任务进入“生成成功”；如果发生插件异常，则迁移到“生成失败”；如果用户取消，则进入“已取消”。生成成功后，系统执行写入场景快照，任务进入“保存版本中”，若保存成功，状态迁移到“已归档”，表示该任务结果已经形成可追踪版本；若保存失败，则进入“生成失败”，因为结果虽然生成出来，但没有可靠沉淀。

我们还给“生成失败”设置了两类后续路径：一是通过重试回到“等待执行”，用于处理临时插件异常或资源恢复后的重新执行；二是通过放弃任务进入“已取消”，表示用户或管理员决定不再继续处理，最终状态可以从“已归档”或“已取消”进入终止节点。这一状态设计明确限制任务不能越过校验直接生成，也不能在失败后无记录地消失，从而保证任务进度展示、失败排查和版本沉淀的一致性。

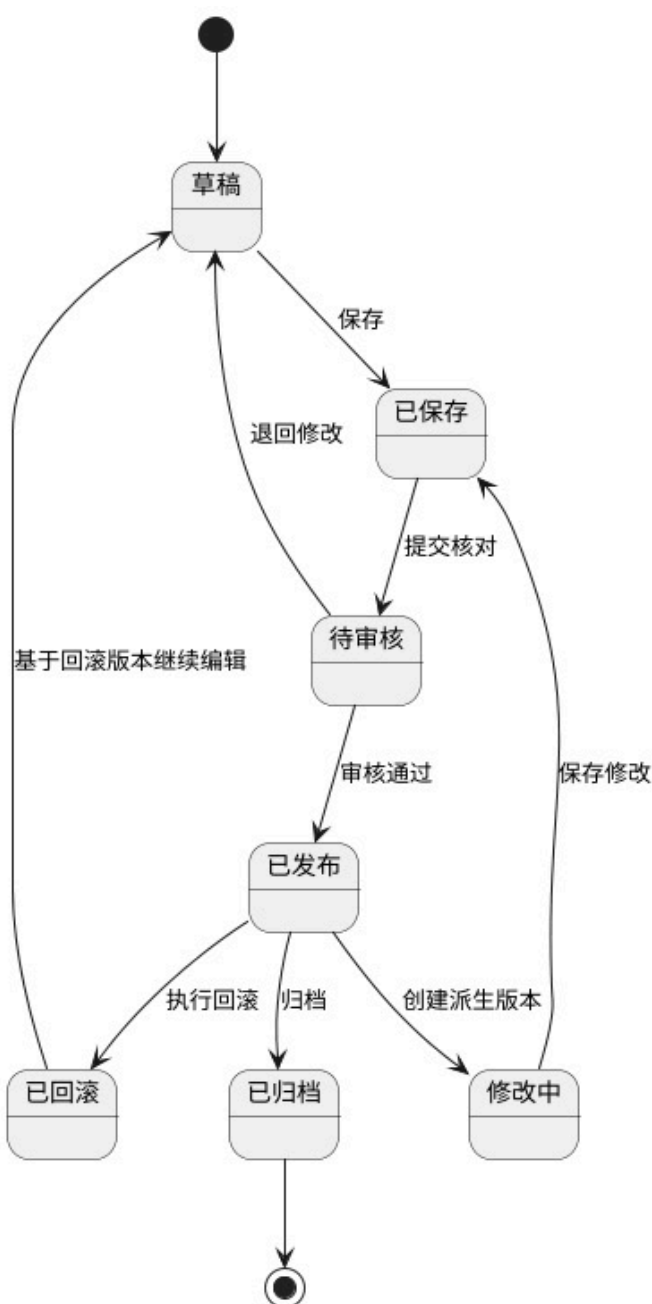


图 21: 场景版本状态图

我们用图 21 描述场景版本从编辑到发布、修改、回滚和归档的生命周期。初始节点进入“草稿”，表示版本仍处于个人编辑或自动生成后的未确认状态。用户执行保存后，版本进入“已保存”；当用户提交核对后，版本进入“待审核”。在“待审核”状态中，如果审核通过，版本迁移到“已发布”；如果需要继续修改，则通过退回修改回到草稿，说明未通过核对的结果不会直接进入发布状态。

我们把“已发布”作为版本生命周期中的核心稳定状态，表示版本可以用于展示、导出或方案比较。已发布版本如果需要进行调整，不直接在原版本上覆盖，而是通过创建派生版本进入“修改中”。修改中版本在保存修改后回到“已保存”，再重新进入提交核对和发布流程。这样的状态设计保证历史已发布版本不被破坏，同时允许在其基础上继续派生新方案。

我们还设置了“已回滚”和“已归档”两个状态。执行回滚后，版本进入“已回滚”；若选择基于回滚版本继续编辑，可以重新回到“草稿”。执行归档后，版本进入“已归档”并到达终止节点，表示该版本不再参与日常编辑。通过这些状态迁移，系统能够清楚区分草稿、已保存、待审核、已发布、修改中、已回滚和已归档等版本阶段，避免多人协作时直接覆盖重要成果。

7.5 状态迁移说明

状态迁移设计的重点是失败恢复和版本保护：生成任务失败时，系统不应覆盖已有场景，而应保留失败原因并允许用户重试；场景版本发布前应处于可编辑或待审核状态，发布后若需要修改，应生成新版本而不是直接覆盖旧版本。

生成任务状态图的价值在于约束任务执行过程。任务不能从“待校验”直接跳到“已归档”，也不能在“生成中”状态下被重复提交。每一次状态迁移都应伴随明确事件，例如校验通过、插件开始执行、插件返回结果、用户取消或系统管理员归档，在后续实现阶段可以将这些状态作为数据库字段和接口返回值，使前端能够展示准确的任务进度。

场景版本状态图则强调成果治理。草稿版本适合个人编辑，待审核版本适合提交给行业分析师核对，已发布版本适合导出或展示，已归档版本适合长期保存。回滚操作不应简单删除当前版本，而应保留回滚记录，说明回滚来源和操作人，这样可以保证项目历史可追踪，避免在多人协作中出现无法判断谁修改了场景的问题。

8 构件图

8.1 构件图简介

构件图（Component Diagram）用于描述软件系统中可独立部署、替换或复用的软件构件，以及这些构件之间的接口和依赖关系。与类图相比，构件图的粒度更粗，它不关心单个类的属性和方法，而是关注系统由哪些服务、模块、接口和外部组件组成。

智能城市生成系统涉及前端交互、后端业务、资源管理、模板管理、任务调度、插件执行、智能解析、动态仿真和日志监控等多个软件单元。如果这些软件单元缺乏清晰边界，扩展新模板、新插件或新智能模型时就会影响主流程稳定性，因此需要用构件图说明系统模块化设计方案。

构件图中的构件通常代表可独立开发、测试、部署或替换的软件单元。在本系统中，资源服务、模板服务、任务服务和 Blender 插件适配服务都适合独立设计，因为它们变化的原因不同：资源服务更多受到文件格式和存储策略影响，模板服务受到生成规则影响，任务服务受到队列和调度策略影响，Blender 插件适配服务则受到 Blender 插件接口变化影响，将这些能力拆成构件，可以降低系统维护成本，也能让道路生成、建筑组装、绿化铺设等能力通过统一编排接入主系统。

表 13: 构件图常用元素说明

元素	图中含义	在本系统中的示例
构件	可独立实现或部署的软件单元	任务服务、资源服务、Blender 插件适配服务
接口	构件对外提供或依赖的服务契约	资源查询接口、插件调用接口、版本保存接口
依赖	一个构件使用另一个构件的能力	任务服务依赖模板服务和资源服务
外部构件	不属于主系统但被系统调用的能力	LLM 服务、Blender 插件执行环境

8.2 构件图设计思路

构件图用于描述系统的软件构件及其依赖关系。智能城市生成系统可以划分为前端工作台、后端业务服务、资源与数据存储、智能交互服务、Blender 插件适配服务、Blender 城市场景插件、仿真服务和监控服务等构件。构件图的重点不是展示类级细节，而是说明这些软件单元如何协同组成完整系统。

8.3 系统构件划分

系统构件划分遵循“职责单一、接口清晰、便于扩展”的原则。前端工作台负责交互；认证与权限服务负责身份认证；项目服务负责项目和场景组织；资源服务负责资产上传和检索；模板服务负责生成规则管理；任务服务负责异步调度；智能交互服务负责自然语言和草图解析；Blender 插件适配服务负责调用 Blender 插件系统；Blender 城市场景插件负责执行具体生成逻辑；版本与导出服务负责成果沉淀；日志监控服务负责运行观测。

表 14: 系统主要构件说明

构件	主要职责	依赖对象
前端工作台	提供登录、项目浏览、参数配置、场景预览和插件入口	API 网关、认证服务
任务服务	创建、调度和跟踪生成任务	模板服务、资源服务、Blender 插件适配服务
智能交互服务	解析自然语言和草图输入	LLM 服务、任务服务
Blender 插件适配服务	统一封装 Blender 插件调用	Blender 城市场景插件、日志监控服务
版本与导出服务	保存场景快照、导出结果并支持回滚	数据库、对象存储

8.4 构件图构建

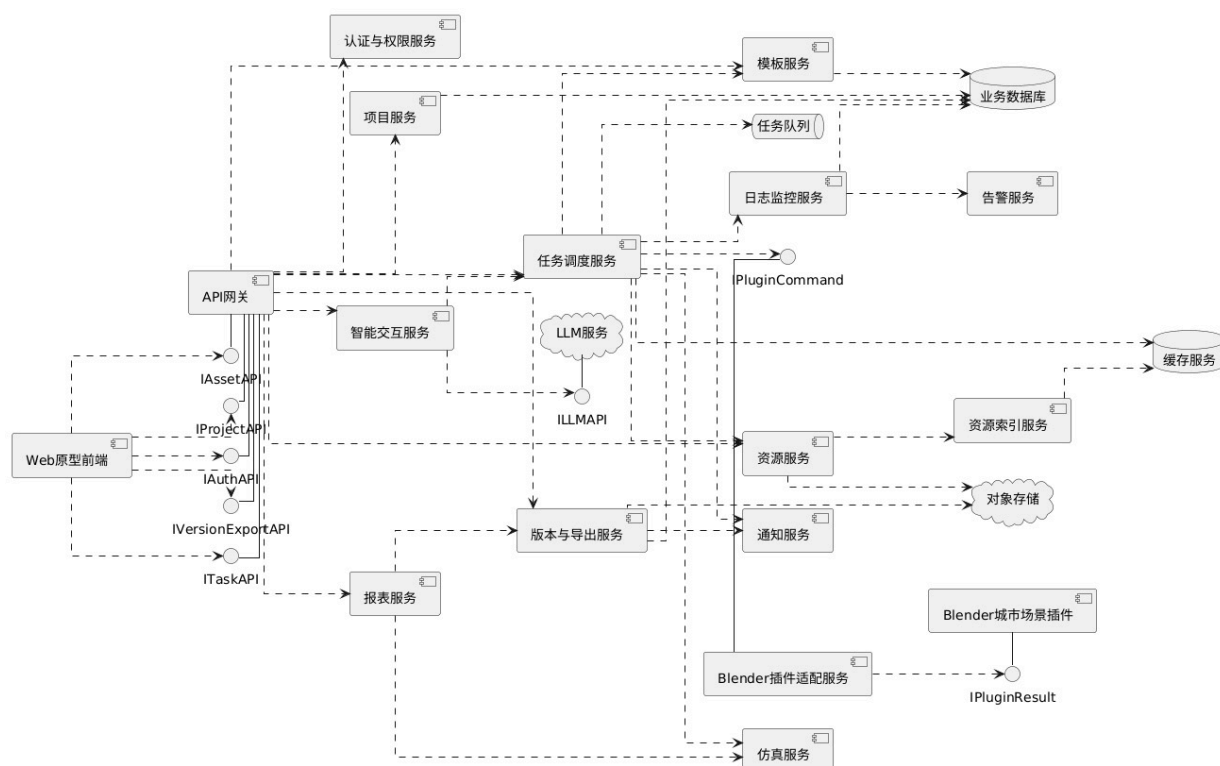


图 22: 智能城市生成系统构件图

我们在图 22 中展示智能城市生成系统的整体构件结构。左侧 Web 原型前端不直接访问具体后端服务，而是通过 API 网关暴露的 `IAssetAPI`、`IProjectAPI`、`IAuthAPI`、`IVersionExportAPI` 和 `ITaskAPI` 等接口进入系统。API 网关之后连接认证与权限服务、项目服务、智能交互服务、任务调度服务、资源服务、版本与导出服务和报表服务等业务构件，说明前端只面向统一入口，具体业务能力由后端服务拆分承担。

中间的任务调度服务是生成链路的编排中心，它依赖模板服务读取模板规则，依赖任务队列处理异步任务，依赖资源服务和资源索引服务检查资源，依赖 Blender 插件适配服务调用插件。智能交互服务通过 `ILLMAPI` 接入 LLM 服务，用于自然语言或草图语义解析；Blender 插件适配服务通过 `IPluginCommand` 调用 Blender 城市市场景插件，并通过 `IPluginResult` 接收插件结果。我们这样处理接口，是为了说明 LLM 和 Blender 插件都是外部可替换构件，主系统只依赖其接口契约。

右侧的数据与运维构件包括业务数据库、缓存服务、对象存储、日志监控服务、告警服务和通知服务。资源服务将大文件写入对象存储、模板和项目等结构化数据写入业务数据库，日志监控服务向告警服务输出异常信息，通知服务负责把任务完成或失败状态推送给用户。综上，我们用构件图明确表达了系统分层：前端负责交互，API 网关负责统一入口，业务构件负责任务编排和项目管理，外部智能与插件构件负责专项处理，数据与监控构件负责长期存储和运行保障。

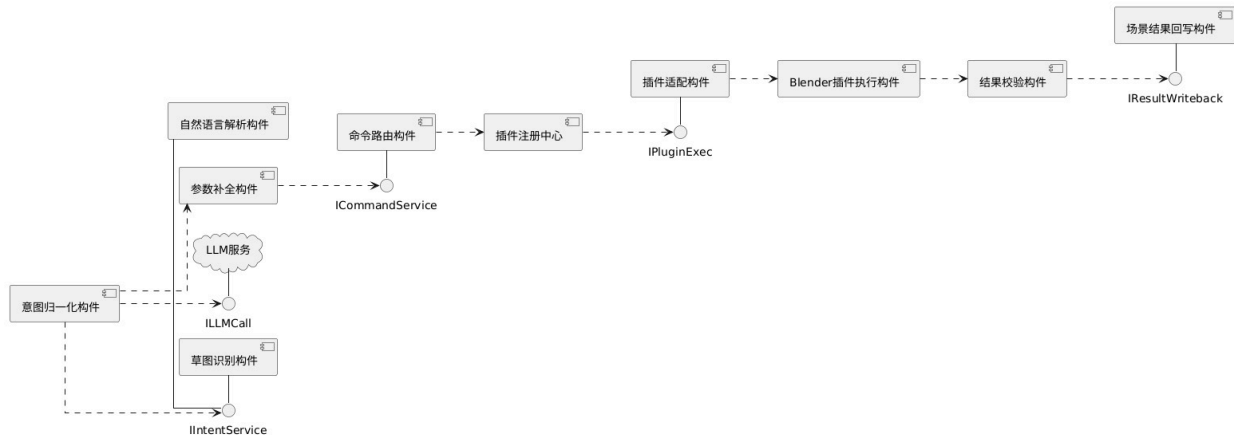


图 23: 插件协同与智能交互构件图

我们用图 23 对智能交互和插件协同这部分作进一步的局部放大。左侧意图归一化构件通过 `IIntentService` 接入自然语言解析构件、参数补全构件和草图识别构件。自然语言解析构件和参数补全构件又通过 `ILLMCall` 访问 LLM 服务，说明大语言模型主要负责语义理解和缺失参数推断，而不直接参与插件执行。

当非结构化输入被归一化后，命令路由构件通过 `ICommandService` 接收结构化命令，并将其交给插件注册中心。插件注册中心保存当前可用插件能力，之后由插件适配构件通过 `IPluginExec` 调用 Blender 插件执行构件，执行结果不会直接回写系统，而是先进入结果校验构件，用于判断返回对象、文件路径和生成数量是否符合任务要求。

最右侧的场景结果回写构件通过 `IResultWriteback` 接收经过校验的结果，并把道路、建筑、绿化、水体等对象写入场景与版本体系。这个局部构件的设计主要强调了智能交互链路中的职责分离：意图归一化解决用户想做什么，命令路由解决该交给哪个能力处理，插件适配解决如何调用 Blender 插件，结果校验和回写解决如何把插件结果安全纳入主系统。

8.5 构件依赖关系说明

构件依赖关系中，前端工作台不直接访问数据库或 Blender 插件，而是通过 API 网关调用后端提供的服务接口。任务服务依赖模板服务、资源服务和插件适配服务；插件适配服务依赖 Blender 城市场景插件；版本与导出服务依赖对象存储和数据库，通过这样的构件拆分，系统可以在不大幅修改主流程的情况下增加新模板、新插件或新的智能解析策略。

构件图还体现了系统的分层结构。前端工作台属于表现层，主要负责用户交互、角色登录和插件入口展示；API 网关和后端服务属于业务层，负责权限校验、流程编排和数据处理；数据库与对象存储属于数据层，负责结构化数据和大文件保存；LLM 服务和 Blender 城市场景插件属于外部能力层，为主系统提供智能解析和三维生成能力。

这种分层设计使系统具有较好的可维护性。若需要替换大语言模型，只需要调整智能交互服务与 LLM 服务之间的接口；若需要增加新的城市生成插件能力，只需要在 Blender 插件适配服务中注册新的插件能力；若需要增加移动端入口，也可以继续复用 API 网关和后端服务，而不必改动核心生成逻辑。

9 部署图

9.1 部署图简介

部署图（Deployment Diagram）用于描述软件系统在物理节点或虚拟节点上的部署结构，它展示系统运行时依赖哪些服务器、终端、数据库、外部服务和网络连接，能够帮助开发人员和运维人员理解系统的运行环境。部署图中的节点可以是实际硬件，也可以是虚拟机、容器、云服务或第三方服务。

智能城市生成系统的部署设计需要考虑三维场景生成的计算开销、模型文件的存储体积、插件执行环境的依赖、智能模型服务的调用方式以及多用户协作访问。与普通信息管理系统相比，该系统对图形处理、文件存储和异步任务执行的要求更高，因此部署图需要明确业务服务、插件执行节点和存储节点之间的边界。

部署图中的节点可以表示物理服务器、虚拟机、容器、云服务或用户设备；而节点之间的连线则表示网络通信路径或依赖关系。部署设计既需要考虑完整部署，也要考虑原型阶段常见的本地轻量化部署，这样做的原因是，智能城市生成系统在早期验证时可以在单机上运行，但在多人协作和大规模资源管理场景下，需要拆分业务服务、数据库、对象存储和插件执行节点。

表 15: 部署图常用元素说明

元素	图中含义	在本系统中的示例
节点	运行软件的物理或虚拟环境	Web 服务器、业务服务器、插件执行节点
制品	部署在节点上的软件或文件	前端应用、后端服务、Blender 插件、模型文件
通信路径	节点之间的数据交换线路	浏览器到 Web 服务的 HTTPS 连接
外部服务	系统运行时依赖的第三方能力	LLM API、对象存储、监控服务

9.2 部署图设计思路

部署图用于描述系统在物理或虚拟运行环境中的分布情况。智能城市生成系统既可以在实验室或演示环境中轻量化部署，也可以在多人协作场景中采用分布式部署。我们在这里主要给出两张部署图：一张描述较完整的系统部署结构，另一张描述本地演示与轻量化部署方式。

9.3 运行节点划分

系统运行节点主要包括用户终端、Web 应用服务器、业务服务服务器、数据库服务器、对象存储节点、Blender 插件执行节点、LLM 服务节点、监控与备份节点。用户终端负责浏览和交互；Web 应用服务器提供前端页面；业务服务器承载主要后端服务；数据库和对象存储保存结构化数据与模型文件；插件执行节点负责运行 Blender 相关任务；LLM 服务节点负责自然语言解析；监控与备份节点负责运行状态观察和数据保护。

表 16: 部署节点与运行内容

部署节点	运行内容	设计要求
用户终端	浏览器、三维预览界面	支持基本图形交互和稳定网络连接
业务服务服务器	后端服务、任务队列、接口服务	需要稳定处理并发请求和异步任务
数据库服务器	用户、项目、任务、版本等结构化数据	保证一致性、备份和权限控制
对象存储节点	模型文件、资源文件、导出结果	支持大文件存储和可追踪引用
插件执行节点	Blender 运行环境和城市生成插件	与普通业务服务隔离，便于控制资源消耗
LLM 服务节点	自然语言解析和指令结构化服务	支持外部 API 或本地模型两种接入方式

9.4 部署图构建

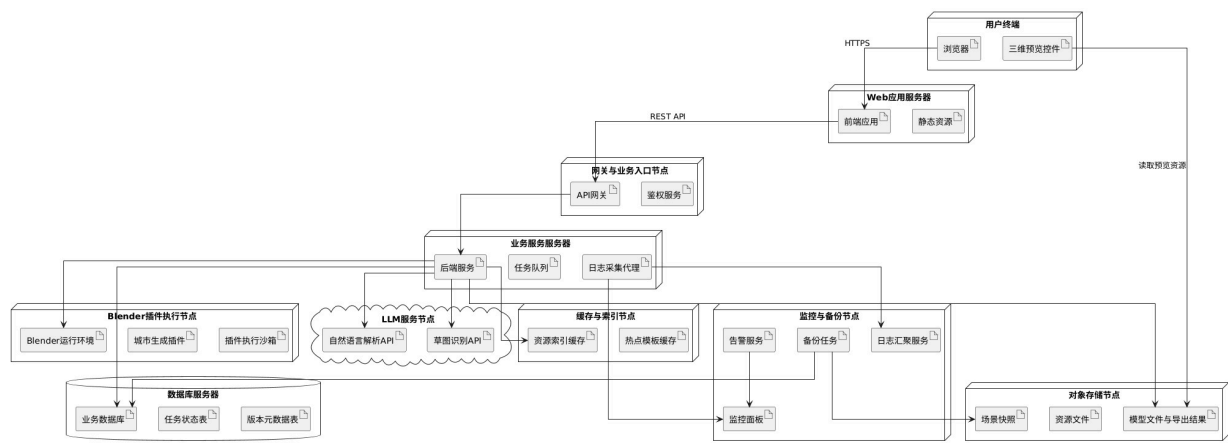


图 24: 智能城市生成系统部署图

我们用图 24 描述多人协作或长期运行场景下的完整部署结构。其中，最上方用户终端包含浏览器和三维预览控件，用户通过 HTTPS 访问 Web 应用服务器，Web 应用服务器内部部署前端应用和静态资源，并通过 REST API 调用网关与业务入口节点。该入口节点包含 API 网关和鉴权服务，用于统一接收请求并进行身份校验。

中部业务服务服务器部署后端服务、任务队列和日志采集代理，是项目管理、任务调度和日志采集的主要运行节点。业务服务一方面连接 Blender 插件执行节点，该节点包含 Blender 运行环境、城市生成插件和插件执行沙箱，用于执行高开销三维生成任务；另一方面连接 LLM 服务节点，调用自然语言解析 API 和草图识别 API 处理智能交互输入。业务服务还访问缓存与索引节点，其中资源索引缓存和热点模板缓存用于减少频繁读取资源元数据和模板规则的开销。

底部和右侧节点负责数据持久化与运维保障。数据库服务器保存业务数据库、任务状态表和版本元数据表；对象存储节点保存场景快照、资源文件、模型文件与导出结果，并供用户端读取预览资源；监控与备份节点包含告警服务、备份任务、日志汇聚服务和监控面板，持续收集运行状态和异常日志。我们通过把 Web 入口、业务服务、插件执行、LLM 服务、数据库、对象存储和监控备份分离，降低了 Blender 任务对普通业务请求的影响，也让大文件存储和结构化数据管理各自独立。

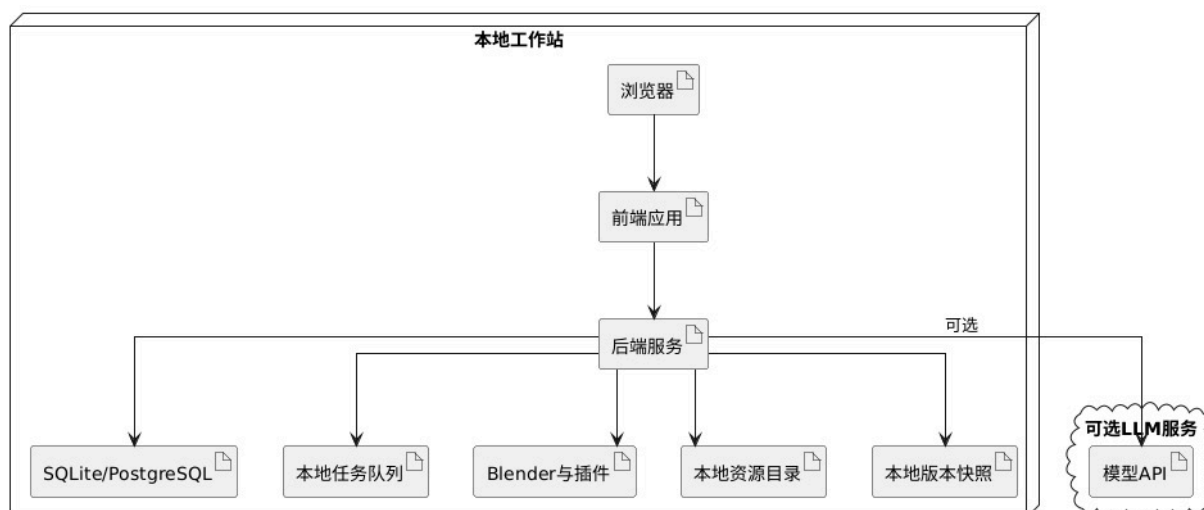


图 25: 本地演示与轻量化部署图

我们用图 25 描述本地演示与轻量化部署方式。其中最大的节点是本地工作站，从上到下依次运行浏览器、前端应用和后端服务，表示用户可以在一台机器上完成访问、交互和业务处理。后端服务向下连接 SQLite/PostgreSQL、本地任务队列、Blender 与插件、本地资源目录和本地版本快照，说明数据库、异步任务、插件执行、资源文件和版本快照都可以部署在同一本地环境中。

右侧的可选 LLM 服务被画在本地工作站之外，并通过模型 API 与后端服务连接，表示演示模式下可以根据网络条件选择接入外部模型服务，也可以关闭该能力，仅使用模板参数和本地插件验证主要流程。与完整部署图相比，轻量化部署没有独立的网关节点、对象存储节点、监控备份节点和插件执行服务器，因此更容易搭建，但资源隔离和并发能力较弱。

这种部署方式适合课程展示、单人原型验证和实验室内部测试，在这种部署下，可以快速跑通创建项目、选择模板、提交生成任务、调用 Blender 插件、保存本地版本快照和导出结果等核心链路；但如果需要多人同时访问、大量模型文件管理或长期日志审计，则应切换到图 24 所示的完整部署模式。

9.5 部署结构说明

部署设计需要兼顾性能、可靠性和可维护性。三维场景生成和插件执行可能消耗较多计算资源，因此插件执行节点应与普通业务服务适当隔离；模型文件和导出结果体积较大，应存入对象存储而不是直接写入业务数据库；对于大语言模型服务，系统应支持外部 API 或本地模型两种接入方式，并通过配置决定使用路径；监控与备份节点用于记录服务状态、任务失败原因和关键数据备份，从而提高系统长期运行的稳定性。

完整部署模式适合多人协作和长期运行。业务服务服务器负责处理用户请求和任务调度，插件执行节点负责运行 Blender 环境，数据库服务器保存结构化数据，对象存储节点保存模型文件和场景快照。这样的拆分可以避免插件任务占满业务服务器资源，也便于对大文件进行独立备份和权限控制。

轻量化部署模式适合演示、单人原型验证或实验室内部测试。在这种模式下，前端应用、后端服务、数据库和 Blender 插件可以运行在同一台工作站上，减少部署成本。虽然这种部署方式不适合高并发和多人协作，但可以快速验证核心业务流程，包括模板生成、自然语言修改、版本保存和结果导出。

10 总结

通过前述各类 UML 图，可以从不同层面较完整地刻画智能城市生成系统的设计结构。用例图明确了场景建模师、行业分析师和系统管理员三类角色的职责边界，并梳理了项目准备、场景生成、结果核对和后台治理等核心功能；活动图进一步展开了生成、核对、异常处理和自然语言修改等关键流程，使资源校验、用户确认和失败回退等环节具备清晰的过程表达。

类图、顺序图和协作图分别从静态结构和动态交互两个角度补充了系统设计。其中，类图说明了项目、场景、资源、模板、任务和版本等核心对象之间的组织关系；顺序图描述了登录、模板驱动生成、自然语言修改和结果导出的消息传递过程；协作图则进一步展示了前端工作台、任务服务、插件适配器、场景服务和版本服务在典型场景中的协同方式。这些设计共同支撑了系统从用户输入到结果沉淀的完整链路。

状态图、构件图和部署图则把设计继续推进到工程实现层面。其中，状态图约束了生成任务和场景版本的生命周期，保证任务重试、版本审核和结果回滚具备明确边界；构件图说明了主系统、智能交互服务和 Blender 插件系统之间的职责划分；部署图进一步给出了多人协作环境与轻量化演示环境下的运行结构。通过这些建模结果，可以为后续数据库设计、接口实现、任务调度、插件接入和系统部署提供较稳定的设计依据。